Applied Software Project Report

E-Commerce Backend Platform with Multi-Gateway Payments and Redis Caching

By

Shivendra Pratap Singh

A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment of the requirements for the degree of Master of Science in Computer Science

October, 2026

Scaler Mentee Email ID : shivendrasahab@gmail.com

Thesis Supervisor : Naman Bhalla

Date of Submission : 11/10/2026

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

.....

Project Guide / Supervisor

Declaration

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 23 November 2023 to October 2026, is the result of my own individual endeavor. The Project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Shivendra Pratap Singh

.....
Signature of the Candidate

Date: 11 October 2026

Acknowledgment

I would like to express my sincere gratitude to everyone who supported me through the Scaler Neovarsity program and the completion of this applied software project.

First, I thank my thesis supervisor, Naman Bhalla, whose lectures on low-level design, Spring Boot and backend architecture shaped the way this project was structured. His emphasis on writing code that is testable, readable and honest about its trade-offs is reflected throughout this report. I am equally grateful to the Scaler instructors and teaching assistants who conducted the backend specialization classes, reviewed assignments and answered doubts at odd hours. The weekly problem sets on Spring Data JPA, Redis, payment gateway integration and system design were the direct foundation of the features built here.

I thank my colleagues at Amazon, where I work as a Software Development Engineer, for the many informal conversations about caching, idempotency and operational safety that influenced the design decisions in this project. Working on production systems alongside this coursework made the academic material tangible.

Finally, and most importantly, I thank my family for their patience and encouragement during the many evenings and weekends spent on this program. Their support made it possible to balance a demanding job with a master's degree.

Any errors or shortcomings in this work are entirely my own.

Shivendra Pratap Singh

Table of Contents

List of Tables 6

List of Figures 7

Applied Software Project

Chapter 1: Abstract 8

Chapter 2: Project Description 9

Chapter 3: Requirement Gathering 13

Chapter 4: Class Diagrams 21

Chapter 5: Database Schema Design 28

Chapter 6: Feature Development Process 33

Chapter 7: Deployment Flow 39

Chapter 8: Technologies Used 43

Chapter 9: Conclusion 48

Chapter 10: References 51

List of Tables

Table No.	Title	Page No.
2.01	Project objectives and how each is met	9
3.01	Functional requirements	13
3.02	Non-functional requirements	14
3.03	System users and their goals	15
3.04	Feature set: authentication and session management	16
3.05	Feature set: catalogue, categories and search	16
3.06	Feature set: cart and orders	18
3.07	Feature set: payments	18
3.08	Standard error responses	19
4.01	Domain entities and their responsibilities	22
4.02	Service classes and their collaborators	23
4.03	Design patterns applied in the project	24
5.01	Indexes defined on the schema	30
5.02	Cardinality of relations	31
5.03	Redis key layout for the popular-products cache	32
6.01	Redis commands issued per request path	35
6.02	Benchmark: latency of GET /products/popular, database vs Redis	37
6.03	Unit tests covering the popular-products cache	38
7.01	VPC subnet plan	39
7.02	Security group rules	40
7.03	Environment properties injected by Elastic Beanstalk	41
7.04	Indicative monthly cost of the production topology	42
8.01	Technology stack summary	43
8.02	Maven dependencies and their purpose	46
9.01	Limitations and suggested improvements	49

List of Figures

Figure No.	Title	Page No.
2.01	High-level architecture of the e-commerce backend	11
2.02	Project development process	11
3.01	Use-case diagram	15
4.01	Class diagram: domain model (JPA entities)	22
4.02	Class diagram: service layer and strategy patterns	23
4.03	Sequence diagram: place order	25
4.04	Sequence diagram: payment initiation and webhook reconciliation	26
5.01	Entity-relationship diagram	28
6.01	Popular-products feature flow: read path and write-through	35
6.02	Benchmark chart: database vs Redis latency	37
7.01	AWS deployment topology	39

Applied Software Project

Chapter 1: Abstract

This project delivers a production-style backend for an online retail store, built with Java 21 and Spring Boot 3.3. The system exposes a REST API that covers the complete shopping journey: browsing and searching a product catalogue, creating an account and logging in, maintaining a per-user shopping cart, converting that cart into an order with stock reservation, and paying for the order through a hosted checkout page provided by either Razorpay or Stripe. The work was undertaken to consolidate the backend specialization of the Scaler Neovarsity Master of Science programme into a single cohesive application, and to practise the engineering judgement that distinguishes a working prototype from a service that can be operated safely.

The implementation follows a strict controller-service-repository layering. Business rules live in transactional service classes; persistence is handled by Spring Data JPA over MySQL (H2 in development and tests); and a Redis read-through cache accelerates the high-demand "popular products" endpoint. Payment providers are abstracted behind a strategy interface so a new gateway is one additional class. Authentication uses BCrypt-hashed passwords and opaque, database-backed session tokens that can be revoked instantly. A single exception handler converts every failure into a uniform JSON error body. The codebase is covered by thirty unit and context tests and a reproducible latency benchmark.

The benchmark shows that serving popular products from Redis reduces median response time from 4.52 ms to 0.99 ms and 95th-percentile latency from 6.55 ms to 1.87 ms, while the cache degrades gracefully to the database if Redis is unavailable. A full AWS deployment design using a VPC, Elastic Beanstalk, RDS for MySQL and ElastiCache for Redis is presented with security-group rules and cost estimates.

Beyond retail, the same building blocks apply to any transactional platform: ticketing, food delivery, subscription billing, or marketplace software. The report documents requirements, low-level design, schema, one feature in depth, deployment, technology rationale and limitations, so that it can serve as a reference for engineers building similar systems.

Chapter 2: Project Description

2.1 Background and motivation

Online commerce is one of the most mature applications of distributed software, yet it remains an excellent teaching domain because it touches almost every backend concern at once. A single purchase requires identity (who is buying), a catalogue (what is for sale), inventory (is it in stock), transactional integrity (do not sell the same unit twice), money movement (a third party must confirm payment), and performance (customers abandon slow pages). Each of these concerns maps directly onto a module of the Scaler backend specialization: Spring Boot and dependency injection, Spring Data JPA and relational modelling, Redis caching, payment-gateway integration, and system design.

The motivation for this project was therefore twofold. First, to assemble the individually studied pieces into one application in which they must co-operate, since the hard problems appear at the seams rather than inside any single component. Second, to build it the way a small engineering team would build a real service: with a reproducible build, automated tests, a clear folder structure, operational health checks, and documentation that explains the trade-offs rather than only the mechanics.

2.2 Problem statement

Design and implement a backend service for an e-commerce store that allows customers to register, browse and search products, manage a shopping cart, place orders against live inventory, and pay through at least two independent payment providers; and that allows catalogue administrators to maintain products and categories. The service must expose a documented REST API, persist its data in a relational database, use an in-memory cache to accelerate hot reads, handle failures predictably, and be deployable to a public cloud with a secure network topology.

2.3 Objectives

The objectives set at the start of the project, and the mechanism by which each one is satisfied in the final system, are listed in Table 2.01.

Table 2.01: Project objectives and how each is met

Objective	How it is achieved
Complete purchase journey through a REST API	Seven controllers covering auth, categories, products, search, cart, orders and payments; 27 endpoints in total.
Secure authentication without heavyweight frameworks	BCrypt password hashing via spring-security-crypto; 256-bit random session tokens stored in a `sessions` table; a single `HandlerInterceptor` guards protected routes.
Correct inventory under concurrent orders	Order placement runs in one database transaction that validates and decrements stock for every line; any failure rolls the whole order back.
Pluggable payment providers	A `PaymentGateway` strategy interface with Razorpay and Stripe implementations; the provider is chosen per request or by configuration.
Low-latency reads for hot products	Redis HASH + SET cache with write-through on catalogue changes and automatic database fallback when Redis is unreachable.

Predictable error handling	`@RestControllerAdvice` maps every domain exception and validation failure to one JSON error shape with the correct HTTP status.
Zero-setup local development	Default profile uses an in-memory H2 database in MySQL mode and seeds ten demo products; Redis is optional.
Verifiable quality	30 automated tests run by `./mvnw test`; a benchmark script reproduces the performance figures in this report.
Cloud deployability	Externalised configuration through environment variables; a documented AWS topology with VPC, Elastic Beanstalk, RDS and ElastiCache.

2.4 Scope

The project is deliberately scoped as a single deployable Spring Boot application rather than a set of microservices. This decision was taken consciously. A monolith with clean internal module boundaries is easier to build, test and reason about for a team of one, and the boundaries drawn here (auth, catalogue, cart, order, payment) are exactly the seams along which the system could later be split if load demanded it. The alternative, several services with their own databases and inter-service calls, would have consumed most of the available effort on infrastructure rather than on the domain logic the project is meant to demonstrate.

In scope: user signup, login, logout and session validation; category and product CRUD with soft deletion; paged and sorted listing; keyword search; a Redis-backed popular-products endpoint; cart management; order placement and cancellation with stock accounting; payment initiation through Razorpay Payment Links or Stripe Checkout Sessions; webhook reconciliation that marks orders as paid; a uniform error model; health endpoint; seeded demo data; and the ability to proxy the public FakeStore API as an alternative catalogue source.

Out of scope: a web or mobile front end, role-based authorisation beyond the data model (the `Role` enum exists but every authenticated user is treated alike), shipping and fulfilment workflows, refunds, promotional pricing, and cryptographic verification of webhook signatures (discussed as future work in Chapter 9).

2.5 System overview

Figure 2.01 shows the high-level architecture. Clients speak HTTP and JSON to an embedded Tomcat server on port 8080. Requests to protected routes pass through `AuthInterceptor`, which resolves the bearer token to a `User`. Controllers validate request bodies and delegate to services. Services own the business rules and transaction boundaries, and talk to Spring Data repositories for persistence, to `RedisTemplate` for the cache, and to the payment-gateway SDKs for checkout links. MySQL holds nine tables; Redis holds two keys.

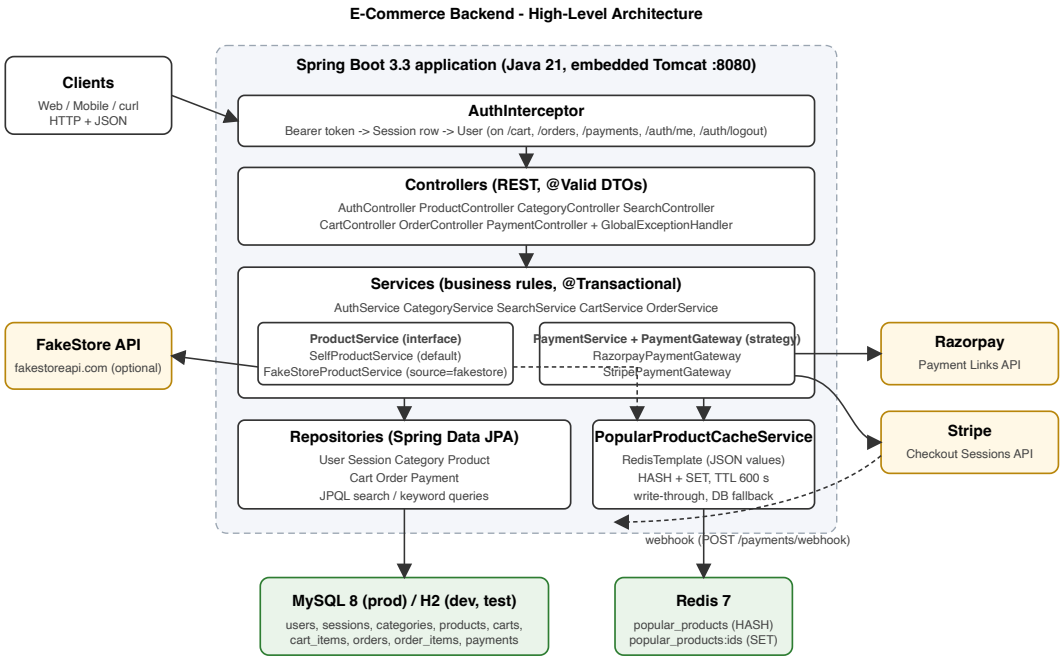


Figure 2.01: High-level architecture of the e-commerce backend

Two configuration switches change the system's behaviour without code changes. `product.source` selects whether the catalogue is served from the project's own tables (`SelfProductService`, the default) or proxied from the public FakeStore API (`FakeStoreProductService`), demonstrating the dependency-inversion principle: controllers depend on the `ProductService` interface, never on an implementation. `payment.default-provider` selects the gateway used when a payment request does not name one.

2.6 Development process

The project followed an incremental, test-as-you-go process illustrated in Figure 2.02. Requirements were written first as the feature tables in Chapter 3. The domain model and schema were designed next, because almost every later decision depended on them. Implementation proceeded one vertical slice at a time: entities and repositories, then the service, then the controller, then unit tests for the service, then a manual smoke test with `curl` against the running application. Each slice was compiled and tested before the next began.

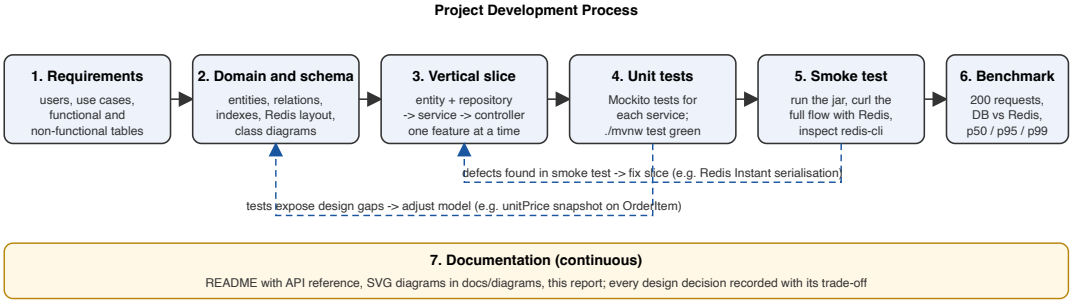


Figure 2.02: Project development process

This discipline paid off concretely. The first end-to-end smoke test with a live Redis instance revealed that the stock Jackson serializer used by Spring Data Redis cannot serialise `java.time.Instant`, so every

cache write was silently failing over to the database. The graceful-degradation design meant the API still returned correct results, which is exactly why the bug would have gone unnoticed without the smoke test. The fix, a dedicated `ObjectMapper` with the JSR-310 module and restricted polymorphic typing, is described in Chapter 6.

2.7 Relevance

The patterns demonstrated here are not specific to retail. Session-token authentication, transactional reservation of a finite resource, strategy-based integration with interchangeable third parties, idempotent webhook handling, and cache-aside with graceful degradation appear in ticket booking, hotel reservation, ride hailing, subscription billing and many internal enterprise systems. Chapter 8 gives real-world examples for each technology. The project is therefore relevant both as a portfolio piece for backend engineering roles and as a reference implementation for teams starting a similar system.

2.8 Repository and deliverables

The complete source code, diagrams, benchmark script and this report are available in the public repository at https://github.com/shivendrasahab/ecommerce-backend. The repository includes a README with setup instructions for the zero-configuration H2 profile, the MySQL and Redis profile via Docker Compose, an API reference with example requests, and the test and benchmark commands. The folder structure follows standard Maven conventions and is reproduced in Chapter 4.

Chapter 3: Requirement Gathering

3.1 Approach

Requirements were gathered by studying the purchase flow of three production retail platforms (Amazon.in, Flipkart and Myntra) as a customer, by reviewing the Razorpay and Stripe integration guides to understand what a payment provider expects from a merchant backend, and by reusing the problem statements of the Scaler backend assignments as a baseline for the catalogue, cart, order and cache modules. Each candidate requirement was written as a short testable statement, classified as functional or non-functional, and assigned a priority: Must (needed for a complete purchase), Should (expected by users but the purchase works without it), and Could (nice to have, deferred if time ran short). Everything marked Must or Should was implemented.

3.2 Functional requirements

Table 3.01: Functional requirements

ID	Requirement	Priority
FR-01	A visitor can create an account with name, unique email, password of at least 8 characters and optional phone number.	Must
FR-02	A registered user can log in with email and password and receive a session token with an expiry time.	Must
FR-03	A logged-in user can log out, immediately invalidating the token.	Must
FR-04	Logging in again expires any previous active session for that user (one active session per user).	Should
FR-05	Anyone can list categories and list products with paging and sorting.	Must
FR-06	Anyone can fetch one product by id; soft-deleted products return 404.	Must
FR-07	Anyone can list the products of a category with paging.	Must
FR-08	Anyone can search products by a keyword matched against title and description, case-insensitively, with paging and sorting by title, price or creation time.	Must
FR-09	Anyone can fetch the list of "popular" products (titles containing configured keywords) served from a cache.	Should
FR-10	A catalogue administrator can create, update and soft-delete products; the category is given by name and created on demand.	Must
FR-11	A catalogue administrator can create categories; duplicate names are rejected.	Must
FR-12	The catalogue can alternatively be served by proxying the public FakeStore API, selected by configuration.	Could
FR-13	A logged-in user has exactly one cart, created on first access.	Must
FR-14	A user can add a product with a quantity; adding the same product again merges quantities.	Must
FR-15	A user can set an absolute quantity for a cart line; zero removes it. A user can empty the cart.	Must
FR-16	Cart quantities are validated against current stock; exceeding stock is rejected with a clear message.	Must
FR-17	A user can place an order from a non-empty cart, supplying a shipping address. The order starts in PENDING_PAYMENT.	Must

FR-18	Placing an order decrements stock for every line, records the unit price at the time of purchase, and empties the cart, atomically.	Must
FR-19	A user can list their orders (newest first, paged) and fetch one of their orders; other users' orders are not visible.	Must
FR-20	A user can cancel a PENDING_PAYMENT order; stock is restored. Paid orders cannot be cancelled by the customer.	Must
FR-21	A user can initiate payment for a PENDING_PAYMENT order and receive a hosted checkout link from Razorpay or Stripe.	Must
FR-22	The gateway can notify the backend of the payment outcome through a webhook; SUCCESS marks the order PAID.	Must
FR-23	Webhook processing is idempotent: repeated deliveries do not change state twice.	Should
FR-24	A user can list the payment attempts for one of their orders.	Should
FR-25	Every error response has the same JSON shape with status, error, message and timestamp.	Must
FR-26	The application exposes a health endpoint suitable for a load balancer.	Should
FR-27	On an empty database the application seeds a demonstration catalogue.	Could

3.3 Non-functional requirements

Table 3.02: Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	Catalogue read endpoints respond within 50 ms at p95 on a single small instance for a catalogue of 10,000 products; the popular-products endpoint responds within 5 ms at p95 when the cache is warm.
NFR-02	Availability	Loss of the Redis cache must not cause request failures; the service falls back to the database and continues to pass its health check.
NFR-03	Consistency	Stock can never go negative and an order's total always equals the sum of its lines at the moment of purchase, regardless of later price changes.
NFR-04	Security	Passwords are stored only as BCrypt hashes. Session tokens have at least 256 bits of entropy from a cryptographically secure generator. Protected routes reject missing, malformed, expired or logged-out tokens with 401.
NFR-05	Security	Sorting parameters supplied by callers are allow-listed so arbitrary columns cannot be probed. Page sizes are capped.
NFR-06	Security	No credentials are stored in source control; gateway keys and database passwords come from environment variables.
NFR-07	Maintainability	Strict controller-service-repository layering; no business logic in controllers; each service has unit tests with mocked collaborators.
NFR-08	Extensibility	Adding a payment provider or an alternative product source requires adding one class, with no edits to existing services or controllers.
NFR-09	Portability	The application runs with zero external dependencies in the default profile (embedded H2) and with MySQL and Redis in the production profile, switched by configuration only.
NFR-10	Observability	Health is exposed at /actuator/health. Cache fallbacks and payment events are logged at WARN and INFO respectively with enough context to debug.
NFR-11	Testability	The full test suite runs in under one minute with no external services.
NFR-12	Deployability	The build produces a single executable jar deployable to AWS Elastic Beanstalk; the deployment topology is documented.

3.4 Users and use cases

Three kinds of actors interact with the system. Table 3.03 summarises them, and Figure 3.01 shows the use-case diagram.

Table 3.03: System users and their goals

Actor	Description	Primary goals
Customer	A shopper, anonymous until they log in. Represented by a `User` row with role CUSTOMER.	Find products quickly, keep a cart across sessions, place an order with confidence that stock is reserved, pay with a familiar provider, see order history, cancel if they change their mind before paying.
Catalogue Admin	A merchant operator. In this version the catalogue endpoints are open so the API can be exercised without a separate admin flow; the `Role` enum and interceptor are the hooks for restricting them.	Add and update products and categories, withdraw products without breaking order history, optionally import an external catalogue.
Payment Gateway (system actor)	Razorpay or Stripe. Called by the backend to create a checkout; calls the backend afterwards to report the outcome.	Receive a well-formed checkout request; deliver the result to a public webhook that accepts duplicate deliveries.
FakeStore API (system actor)	Public catalogue used as an alternative product source for demonstrations.	None; it is a passive provider.

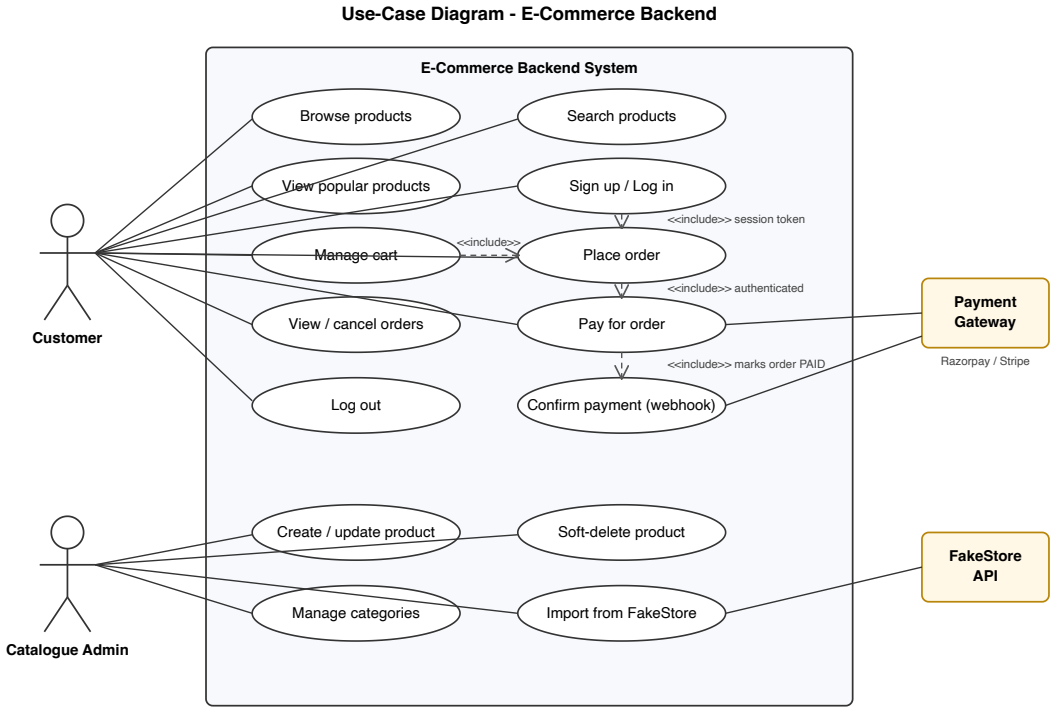


Figure 3.01: Use-case diagram

3.4.1 Use case: place an order

Actor: Customer. **Precondition:** the customer is logged in and has at least one item in the cart. **Main flow:** the customer submits a shipping address; the system loads the cart, verifies that every line's quantity

is still available, decrements stock, copies each product's current price into the order line, records the total, empties the cart and returns the order in state PENDING_PAYMENT. **Alternative flows:** if the cart is empty the system returns 400; if any line exceeds stock the system returns 400 naming the product and no stock is changed; if the token is invalid the system returns 401. **Postcondition:** an `orders` row and its `order_items` exist, stock is reduced, the cart is empty.

3.4.2 Use case: pay for an order

Actor: Customer, then Payment Gateway. **Precondition:** an order in PENDING_PAYMENT belongs to the customer. **Main flow:** the customer requests payment, optionally naming a provider and a callback URL; the system asks the gateway for a hosted checkout link, stores a `payments` row in state INITIATED with the gateway's reference id, and returns the link. The customer completes payment on the gateway's page. The gateway posts the outcome to the webhook; the system finds the payment by reference id, marks it SUCCESS or FAILED, and on SUCCESS marks the order PAID. **Alternative flows:** if the order is not PENDING_PAYMENT, 400; if the gateway rejects the request (for example, invalid keys), 502 and no payment row is written; if the webhook is delivered twice, the second delivery is a no-op.

3.4.3 Use case: view popular products

Actor: Customer (anonymous allowed). **Main flow:** the system reads the set of popular product ids and their JSON bodies from Redis in two commands and returns them. **Alternative flows:** if the keys are absent (first request or expired TTL) the system queries the database by keyword, returns the result and warms the cache; if Redis is unreachable the system logs a warning and returns the database result.

3.5 Feature set

The following tables list every endpoint grouped by module, with its authentication requirement and the service method that implements it. "Bearer" means the request must carry `Authorization: Bearer <token>`.

Table 3.04: Feature set: authentication and session management

Method	Path	Auth	Behaviour	Implementation
POST	/auth/signup	-	Validates body, rejects duplicate email (400), hashes password, stores user with role CUSTOMER. Returns 201 and the user without the hash.	AuthService.signup
POST	/auth/login	-	Verifies password against BCrypt hash. Expires previous ACTIVE sessions. Creates a session with a 48-hour TTL. Returns token, email, name, expiresAt.	AuthService.login
POST	/auth/logout	Bearer	Marks the session LOGGED_OUT. Returns 204.	AuthService.logout
GET	/auth/me	Bearer	Returns the authenticated user.	AuthInterceptor + CurrentUser

Table 3.05: Feature set: catalogue, categories and search

Method	Path	Auth	Behaviour	Implementation

GET	/categories	-	All categories.	CategoryService.getAllCategories
GET	/categories/{id}	-	One category or 404.	CategoryService.getCategoryById
POST	/categories	-	Create; duplicate name is 400.	CategoryService.createCategory
GET	/products	-	Paged listing; query params page, size, sortBy, direction.	ProductService.getAllProducts
GET	/products/popular	-	Cached popular products (see Chapter 6).	PopularProductCacheService
GET	/products/{id}	-	One product; soft-deleted is 404.	ProductService.getProductById
GET	/products/category/{id}	-	Products of a category, paged; unknown category is 404.	ProductService.getProductsByCategory
POST	/products	-	Create with title, price, stock, category name (created if absent). Updates cache if popular.	SelfProductService.createProduct
PUT	/products/{id}	-	Full update. Updates or evicts cache entry.	SelfProductService.updateProduct
DELETE	/products/{id}	-	Soft delete (deleted = true). Evicts cache entry. 204.	SelfProductService.deleteProduct

GET	/search	-	q, page, size (max 50), sortBy in {title, price, createdAt}, direction. Case-insensitive LIKE on title and description.	SearchService.search

Table 3.06: Feature set: cart and orders

Method	Path	Auth	Behaviour	Implementation
GET	/cart	Bearer	Current cart, created on first access; includes computed total.	CartService.getOrCreateCart
POST	/cart/items	Bearer	Add productId with quantity; merges with existing line; stock-checked.	CartService.addItem
PUT	/cart/items/{productId}	Bearer	Set absolute quantity (query param); 0 removes; stock-checked; unknown line is 404.	CartService.updateItem
DELETE	/cart/items/{productId}	Bearer	Remove one line.	CartService.removeItem
DELETE	/cart	Bearer	Empty the cart. 204.	CartService.clearCart
POST	/orders	Bearer	Place order from cart with shippingAddress. Transactional stock reservation and price snapshot. 201.	OrderService.placeOrder
GET	/orders	Bearer	Caller's orders, newest first, paged.	OrderService.getOrders
GET	/orders/{orderId}	Bearer	One of the caller's orders; others' orders are 404.	OrderService.getOrder
POST	/orders/{orderId}/cancel	Bearer	Cancel if PENDING_PAYMENT; restores stock; otherwise 400.	OrderService.cancelOrder

Table 3.07: Feature set: payments

Method	Path	Auth	Behaviour	Implementation

POST	/payments/initiate	Bearer	Body orderId, optional provider (RAZORPAY or STRIPE), optional callbackUrl. Creates a hosted checkout and a Payment row in INITIATED. Returns 201 with the link.	PaymentService.initiatePayment; RazorpayPaymentGateway; StripePaymentGateway
GET	/payments/order/{orderId}	Bearer	Payment attempts for one of the caller's orders.	PaymentService.getPaymentsForOrder
POST	/payments/webhook	-	Body providerReferenceId, status (SUCCESS or FAILED). Idempotent reconciliation; SUCCESS marks the order PAID.	PaymentService.handleWebhook; OrderService.markPaid

3.6 Error model

Requirement FR-25 demands one error shape. Every failure, whether raised by validation, by a service, by a missing route or by an unexpected exception, is converted by `GlobalExceptionHandler` into the body shown below, with the status codes in Table 3.08.

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "Only 2 unit(s) of 'Rolex Submariner' available, requested 5",
  "timestamp": "2026-10-06T15:52:23.404911Z"
}
```

Table 3.08: Standard error responses

HTTP	Raised by	Typical cause
400	BadRequestException, InsufficientStockException, MethodArgumentNotValidException	Duplicate email, empty cart, cancelling a paid order, quantity above stock, unknown sort field, failed bean validation (first field error is reported).
401	UnauthorizedException	Missing or malformed Authorization header, unknown token, expired or logged-out session, wrong password.
404	ResourceNotFoundException, NoResourceFoundException	Unknown or soft-deleted product, unknown category, order belonging to another user, unknown route.
502	PaymentGatewayException	Razorpay or Stripe rejected the checkout request (invalid keys, network failure).
500	Exception (catch-all)	Any unanticipated error; the message is included for debugging in this academic build and would be redacted in production.

3.7 Assumptions and constraints

- Currency is Indian Rupees throughout; gateways receive amounts in paise (amount x 100, rounded).
- Inventory is a single integer per product with no warehouse or variant dimension.

- Webhook payloads are simplified to the two fields the backend needs; a production integration would parse the provider's full event and verify its signature.
- The system is designed to run as multiple identical instances behind a load balancer; no in-process state is relied upon (sessions live in the database, cache in Redis).
- The project targets Java 21 and Spring Boot 3.3.5 and uses the Maven wrapper so no local Maven installation is needed.

Chapter 4: Class Diagrams

4.1 Package structure

The source tree follows the standard Maven layout with one Java package per architectural layer under `com.shivendra.ecommerce`. The layering rule is one-directional: controllers may depend on services and DTOs; services may depend on repositories, models and other services; repositories depend only on models. Nothing depends on a controller.

```
src/main/java/com/shivendra/ecommerce
├── EcommerceBackendApplication.java
├── config/          RedisConfig, WebConfig, RestTemplateConfig, PaymentGatewayConfig,
DataSeeder
├── controllers/     AuthController, CategoryController, ProductController,
SearchController,
│                    CartController, OrderController, PaymentController
├── dtos/            SignupRequestDto, LoginRequestDto, LoginResponseDto,
ProductRequestDto,
│                    CategoryRequestDto, CartItemRequestDto, PlaceOrderRequestDto,
│                    InitiatePaymentRequestDto, PaymentResponseDto, PaymentWebhookDto,
│                    PaymentLinkResult, FakeStoreProductDto, ErrorResponseDto
├── exceptions/      ResourceNotFoundException, BadRequestException,
InsufficientStockException,
│                    UnauthorizedException, PaymentGatewayException,
GlobalExceptionHandler
├── models/          BaseModel, User, Role, Session, SessionStatus, Category, Product,
│                    Cart, CartItem, Order, OrderStatus, OrderItem,
│                    Payment, PaymentProvider, PaymentStatus
├── repositories/    UserRepository, SessionRepository, CategoryRepository,
ProductRepository,
│                    CartRepository, OrderRepository, PaymentRepository
├── security/        AuthInterceptor, CurrentUser
└── services/        AuthService, CategoryService, SearchService, CartService,
OrderService,
                     ProductService (interface), SelfProductService,
FakeStoreProductService,
                     PopularProductCacheService
    └── payment/     PaymentGateway (interface), RazorpayPaymentGateway,
                     StripePaymentGateway, PaymentService
```

4.2 Domain model

Figure 4.01 shows the JPA entities. Every entity extends `BaseModel`, a `@MappedSuperclass` providing the surrogate key, creation and update timestamps (populated by Hibernate's `@CreationTimestamp` and `@UpdateTimestamp`), and a `deleted` flag for soft deletion. Keeping these four columns in one place guarantees that every table has them and that the repositories can rely on them.

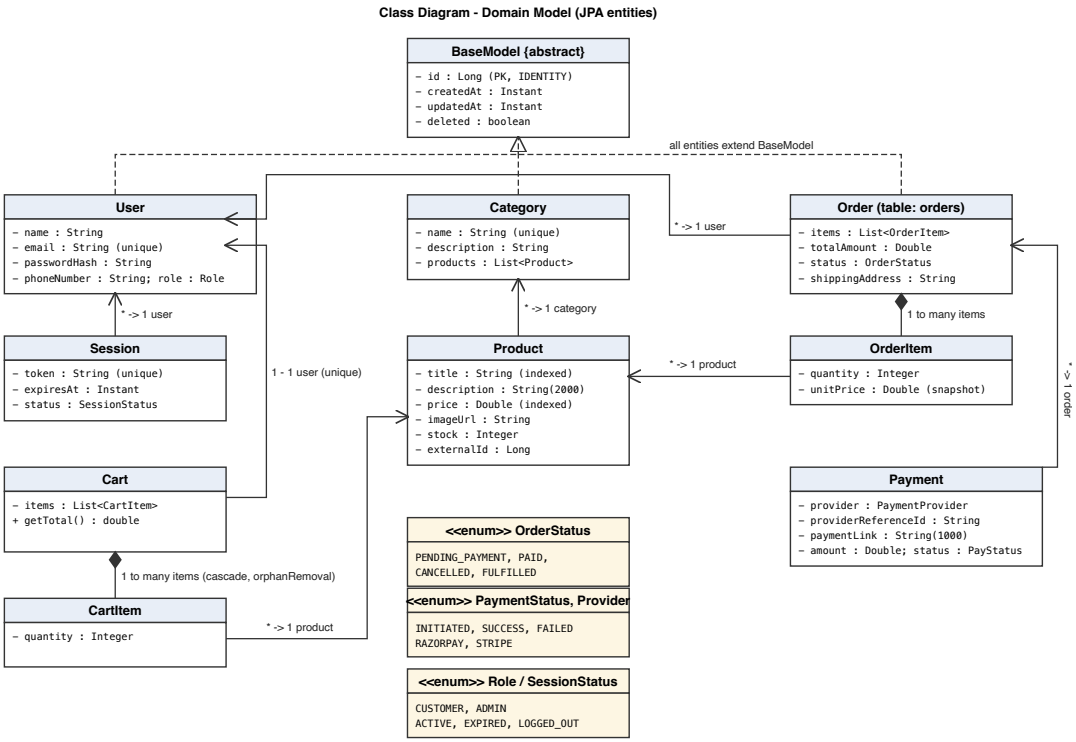


Figure 4.01: Class diagram: domain model (JPA entities)

Table 4.01: Domain entities and their responsibilities

Entity	Responsibility and notable design choices
User	Identity. Email is unique and stored lower-cased. `passwordHash` is annotated `@JsonIgnore` so it can never leak into a response even if the entity is returned directly. `role` defaults to CUSTOMER.
Session	One login. Holds the opaque token (unique, 43 characters of URL-safe base64), expiry instant and a status of ACTIVE, EXPIRED or LOGGED_OUT. Storing sessions in the database makes logout and forced expiry a row update, which is simpler and more revocable than stateless JWTs for a single backend.
Category	Catalogue grouping with a unique name. The reverse `products` collection is `@JsonIgnore`d to avoid infinite recursion and accidental loading of thousands of rows.
Product	Sellable item with title, description, price, image, stock and category. Indexed on title, category and price because those are the columns the search, category and sort queries filter on. `externalId` records the FakeStore id for idempotent imports. Implements `Serializable` because instances are cached in Redis.
Cart	One per user (unique foreign key). Owns its items with cascade ALL and `orphanRemoval`, so removing an item from the Java list deletes the row. `getTotal()` is derived, never stored, so it cannot drift from the lines.
CartItem	A product and quantity inside a cart. The back-reference to the cart is lazy and `@JsonIgnore`d.
Order	Mapped to table `orders` because `order` is an SQL reserved word. Holds the user, lines, total, status and shipping address. Status machine: PENDING_PAYMENT to PAID (via webhook) or CANCELLED (by customer); FULFILLED is reserved for a future fulfilment module.
OrderItem	Snapshot of a purchase line. `unitPrice` is copied from the product at order time so a later price change never alters historical orders (requirement NFR-03).

Payment	One attempt to pay for an order through one provider. Stores the provider's reference id (used to match webhooks) and the hosted link. An order can have several payments (a failed attempt followed by a successful one).

4.3 Service layer

Figure 4.02 shows the service classes and the two interfaces that give the system its pluggability. Services are Spring singletons with constructor injection; every dependency is a final field, which makes them trivial to instantiate in unit tests with Mockito mocks.

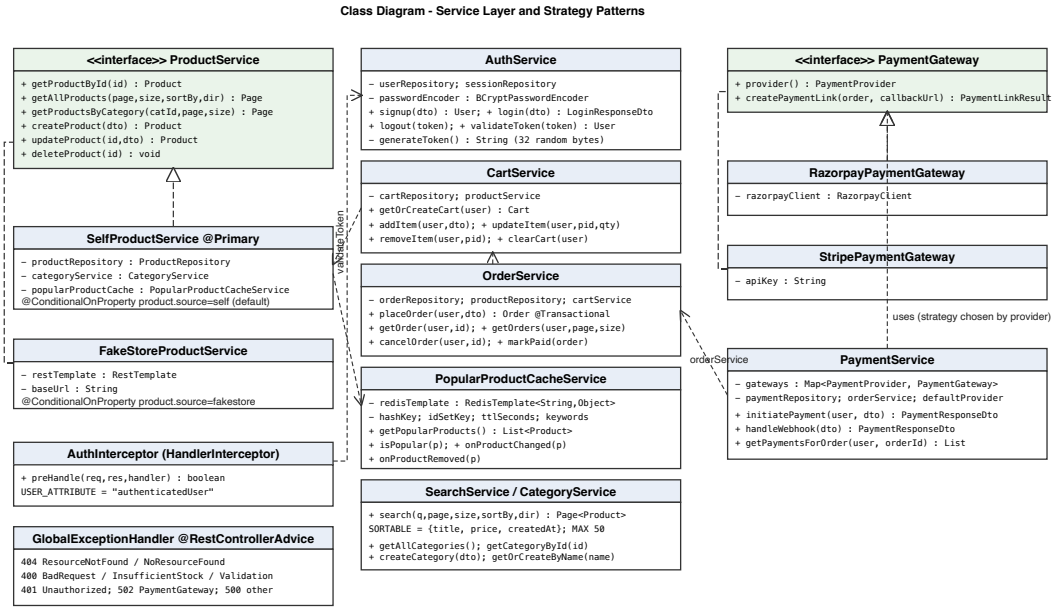


Figure 4.02: Class diagram: service layer and strategy patterns

Table 4.02: Service classes and their collaborators

Service	Collaborators	Responsibility
AuthService	UserRepository, SessionRepository, BCryptPasswordEncoder	Signup, login, logout, token validation. Generates tokens from 32 bytes of `SecureRandom`. Enforces one active session per user and lazily expires stale sessions on validation.
ProductService (interface)	-	Catalogue contract used by controllers and by CartService. Two implementations below.
SelfProductService	ProductRepository, CategoryService, PopularProductCacheService	Default implementation (`@Primary`, active when `product.source=self` or unset). Soft-deletes. Calls the cache after every write so Redis never serves a stale price.
FakeStoreProductService	RestTemplate	Active when `product.source=fakestore`. Maps FakeStore JSON to the `Product` entity. Demonstrates the Adapter pattern over a third-party API.
CategoryService	CategoryRepository	CRUD plus `getOrCreateByName`, used when products arrive with a category string.

SearchService	ProductRepository	Validates the query, allow-lists the sort field, clamps page size to 50, delegates to a JPQL query.
PopularProductCacheService	RedisTemplate, ProductRepository	Read-through cache with write-through updates and database fallback. Detailed in Chapter 6.
CartService	CartRepository, ProductService	Lazy cart creation, quantity merging, absolute updates, stock validation.
OrderService	OrderRepository, ProductRepository, CartService	Transactional order placement, listing, cancellation with stock restoration, and `markPaid` for the payment flow.
PaymentGateway (interface)	-	Strategy contract: `provider()` and `createPaymentLink(order, callbackUrl)`.
RazorpayPaymentGateway	RazorpayClient	Builds a Payment Link request with amount in paise, customer details, reference id and optional callback; returns the short URL.
StripePaymentGateway	Stripe static client	Builds a Checkout Session in payment mode with one line item per order line priced in INR; returns the session URL.
PaymentService	List<PaymentGateway>, PaymentRepository, OrderService	Indexes gateways by provider at startup, validates the order state, delegates link creation, persists the Payment, reconciles webhooks idempotently.

4.4 Design patterns applied

Table 4.03: Design patterns applied in the project

Pattern	Where	Why
Strategy	PaymentGateway with Razorpay and Stripe implementations; PaymentService selects one at runtime	The algorithm for "create a checkout" varies by provider while the surrounding flow (validate order, persist attempt) does not. New providers are added without modifying PaymentService (open-closed principle).
Dependency Inversion	ProductService interface; controllers and CartService depend on it, not on SelfProductService	Allows the FakeStore proxy to replace the database implementation by configuration, and allows CartService to be unit-tested against a mock.
Adapter	FakeStoreProductService	Translates the external FakeStore JSON shape into the internal Product entity so the rest of the system is unaware of the third party.
Template / Mapped Superclass	BaseModel	Shared columns and behaviour for every entity without table inheritance.
Interceptor (Chain of Responsibility)	AuthInterceptor registered in WebConfig	Cross-cutting authentication applied declaratively to path patterns; controllers stay free of security code.
Cache-Aside with Write-Through	PopularProductCacheService	Reads check the cache first and populate it on a miss; writes update the cache immediately so it never serves stale data within the TTL.

Registry (self-registration)	PaymentService constructor receives List<PaymentGateway>	Spring injects every bean implementing the interface; the map from provider to gateway is built automatically, so there is no central list to maintain.
DTO	dtos package	Request bodies carry validation annotations and never expose entity internals; response records (LoginResponseDto, PaymentResponseDto) shape exactly what the client needs.
Centralised exception handling	GlobalExceptionHandler	One translation point from exceptions to HTTP, so controllers contain no try/catch.

4.5 Key interactions

Two sequence diagrams illustrate the most important collaborations. Figure 4.03 traces order placement from the HTTP request through authentication, the transactional service method, and the repository calls, including the alternative flows for an empty cart and insufficient stock. Figure 4.04 traces the two halves of a payment: the authenticated initiation that produces a hosted link, and the unauthenticated webhook that reconciles the result.

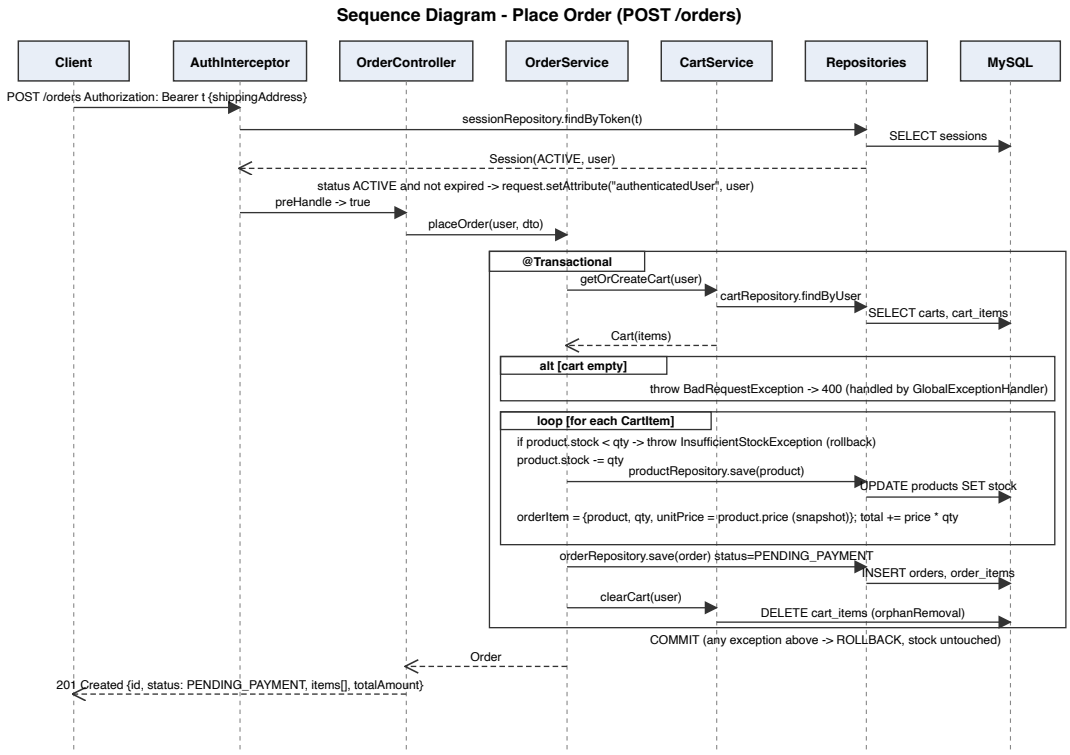


Figure 4.03: Sequence diagram: place order

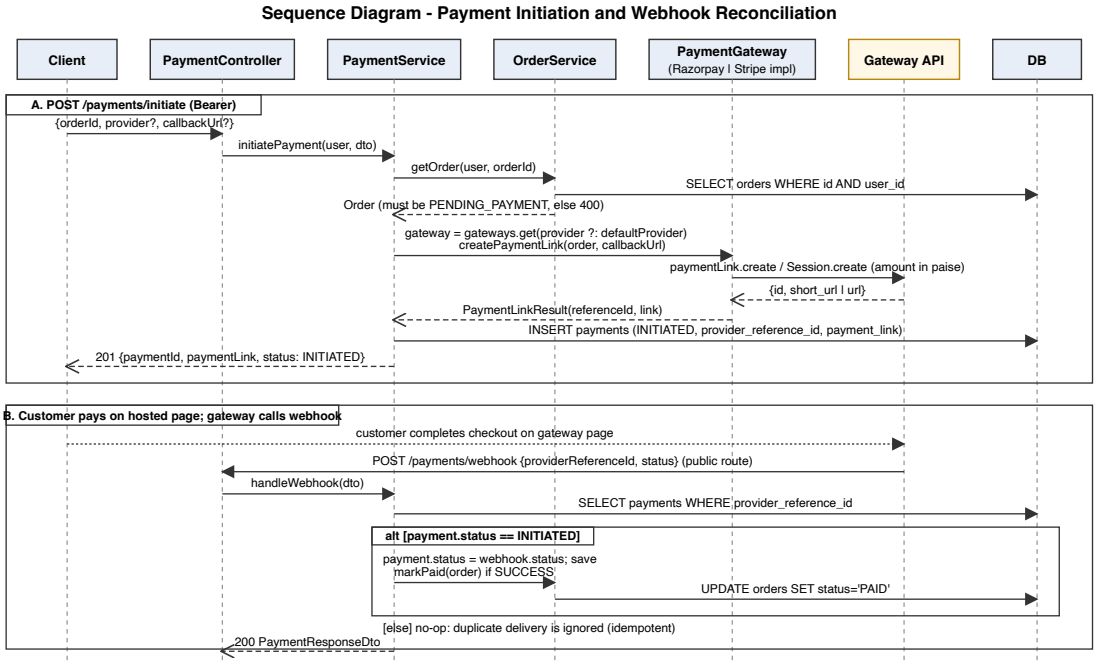


Figure 4.04: Sequence diagram: payment initiation and webhook reconciliation

4.6 Representative code

The strategy interface and the service constructor that indexes its implementations are small enough to show in full. Note that nothing in `PaymentService` names a concrete gateway.

```
public interface PaymentGateway {
    PaymentProvider provider();
    PaymentLinkResult createPaymentLink(Order order, String callbackUrl);
}

@Service
public class PaymentService {
    private final Map<PaymentProvider, PaymentGateway> gateways = new EnumMap<>
(PaymentProvider.class);
    ...
    public PaymentService(List<PaymentGateway> gatewayBeans,
                          PaymentRepository paymentRepository,
                          OrderService orderService,
                          @Value("${payment.default-provider:RAZORPAY}")
PaymentProvider defaultProvider) {
        gatewayBeans.forEach(g -> gateways.put(g.provider(), g));
        ...
    }
}
```

The order-placement method shows the transaction boundary and the price snapshot. Because the method is annotated `@Transactional`, the `InsufficientStockException` thrown mid-loop rolls back any stock decrements already applied to earlier lines.

```
@Transactional
public Order placeOrder(User user, PlaceOrderRequestDto request) {
    Cart cart = cartService.getOrCreateCart(user);
    if (cart.getItems().isEmpty()) {
        throw new BadRequestException("Cannot place an order with an empty cart");
    }
    Order order = new Order();
    order.setUser(user);
    order.setShippingAddress(request.getShippingAddress());
    order.setStatus(OrderStatus.PENDING_PAYMENT);
    double total = 0;
    for (CartItem cartItem : cart.getItems()) {
        Product product = cartItem.getProduct();
        int quantity = cartItem.getQuantity();
        if (product.getStock() < quantity) {
            throw new InsufficientStockException("Only " + product.getStock() + "
unit(s) of '"
                    + product.getTitle() + "' left, cart has " + quantity);
        }
        product.setStock(product.getStock() - quantity);
        productRepository.save(product);
        OrderItem item = new OrderItem();
        item.setOrder(order);
        item.setProduct(product);
        item.setQuantity(quantity);
        item.setUnitPrice(product.getPrice());     // snapshot, not a reference
        order.getItems().add(item);
        total += product.getPrice() * quantity;
    }
    order.setTotalAmount(round2(total));
    Order saved = orderRepository.save(order);
    cartService.clearCart(user);
    return saved;
}
```

Chapter 5: Database Schema Design

5.1 Overview

The relational schema has nine tables generated by Hibernate from the entity annotations (`spring.jpa.hibernate.ddl-auto=update` in development; a production deployment would freeze the DDL with Flyway, see Chapter 9). MySQL 8 is the production engine; H2 in MySQL compatibility mode is used for local development and tests so that the same JPQL and DDL work in both. All tables share four columns inherited from `BaseModel`: `id BIGINT AUTO_INCREMENT PRIMARY KEY`, `created_at TIMESTAMP`, `updated_at TIMESTAMP` and `deleted BOOLEAN`. Figure 5.01 shows the entity-relationship diagram.

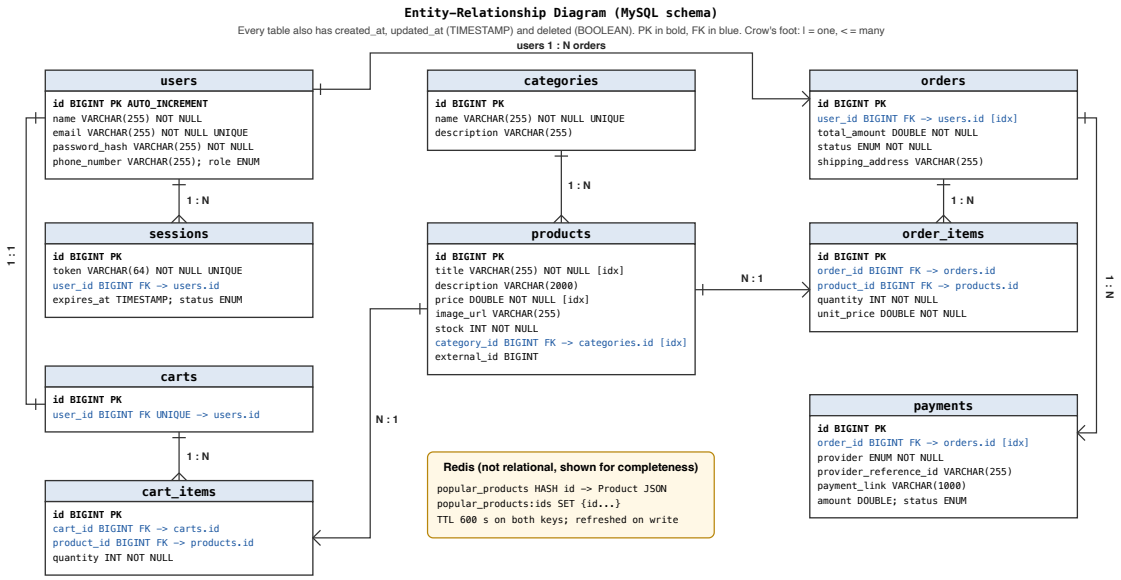


Figure 5.01: Entity-relationship diagram

5.2 Textual schema

```
Tables:

users
- id                      BIGINT, auto-increment
- name                    VARCHAR(255) NOT NULL
- email                   VARCHAR(255) NOT NULL, UNIQUE
- password_hash           VARCHAR(255) NOT NULL        (BCrypt, 60 chars)
- phone_number            VARCHAR(255)
- role                    VARCHAR(255) NOT NULL        (enum: CUSTOMER, ADMIN)
- created_at, updated_at  TIMESTAMP
- deleted                 BOOLEAN NOT NULL
- Primary Key(id)

sessions
- id                      BIGINT, auto-increment
- token                   VARCHAR(64) NOT NULL, UNIQUE
- user_id                 BIGINT NOT NULL
- expires_at              TIMESTAMP NOT NULL
- status                  VARCHAR(255) NOT NULL        (enum: ACTIVE, EXPIRED,
LOGGED_OUT)
- created_at, updated_at, deleted
- Primary Key(id)

categories
- id                      BIGINT, auto-increment
- name                    VARCHAR(255) NOT NULL, UNIQUE
- description             VARCHAR(255)
- created_at, updated_at, deleted
- Primary Key(id)

products
- id                      BIGINT, auto-increment
- title                   VARCHAR(255) NOT NULL
- description             VARCHAR(2000)
- price                   DOUBLE NOT NULL
- image_url               VARCHAR(255)
- stock                   INT NOT NULL
- category_id             BIGINT
- external_id             BIGINT                       (FakeStore id when imported)
- created_at, updated_at, deleted
- Primary Key(id)

carts
- id                      BIGINT, auto-increment
- user_id                 BIGINT NOT NULL, UNIQUE
- created_at, updated_at, deleted
- Primary Key(id)

cart_items
- id                      BIGINT, auto-increment
- cart_id                 BIGINT NOT NULL
- product_id              BIGINT NOT NULL
- quantity                INT NOT NULL
- created_at, updated_at, deleted
- Primary Key(id)

orders
- id                      BIGINT, auto-increment
- user_id                 BIGINT NOT NULL
- total_amount            DOUBLE NOT NULL
- status                  VARCHAR(255) NOT NULL        (enum: PENDING_PAYMENT, PAID,
CANCELLED, FULFILLED)
- shipping_address        VARCHAR(255)
- created_at, updated_at, deleted
- Primary Key(id)

order_items
- id                      BIGINT, auto-increment
- order_id                BIGINT NOT NULL
```

```
- product_id              BIGINT NOT NULL
- quantity                INT NOT NULL
- unit_price              DOUBLE NOT NULL              (price at purchase time)
- created_at, updated_at, deleted
- Primary Key(id)

payments
- id                      BIGINT, auto-increment
- order_id                BIGINT NOT NULL
- provider                VARCHAR(255) NOT NULL        (enum: RAZORPAY, STRIPE)
- provider_reference_id   VARCHAR(255)                 (plink_... or cs_...)
- payment_link            VARCHAR(1000)
- amount                  DOUBLE NOT NULL
- status                  VARCHAR(255) NOT NULL        (enum: INITIATED, SUCCESS,
FAILED)
- created_at, updated_at, deleted
- Primary Key(id)

Foreign Keys:
- sessions(user_id)       refers users(id)
- products(category_id)   refers categories(id)
- carts(user_id)          refers users(id)
- cart_items(cart_id)     refers carts(id)
- cart_items(product_id)  refers products(id)
- orders(user_id)         refers users(id)
- order_items(order_id)   refers orders(id)
- order_items(product_id) refers products(id)
- payments(order_id)      refers orders(id)
```

5.3 Indexes

Indexes were chosen by reading the queries the repositories actually issue rather than by guessing. Each one in Table 5.01 corresponds to a WHERE, JOIN or ORDER BY clause in the code.

Table 5.01: Indexes defined on the schema

Table	Index	Query it serves
users	idx_users_email (unique)	`findByEmailIgnoreCase` on login and `existsByEmailIgnoreCase` on signup. Uniqueness is also the integrity constraint for FR-01.
sessions	token (unique)	`findByToken` on every authenticated request. This is the hottest lookup in the system.
products	idx_products_title	Prefix of the keyword search and the title sort. A leading-wildcard LIKE cannot use it for seeking, but the sort and the FakeStore import by title do.
products	idx_products_category	`findByCategoryIdAndDeletedFalse` for the category listing.
products	idx_products_price	Sort by price in listing and search.
carts	user_id (unique)	`findByUser` on every cart operation; uniqueness enforces one cart per user.
orders	idx_orders_user	`findByUser` (order history) and `findByIdAndUser` (ownership check).
payments	idx_payments_order	`findByOrder` for the payment history endpoint.
payments	provider_reference_id	`findByProviderReferenceId` on every webhook. Recommended as an explicit index in production; currently a plain column.

5.4 Cardinality of relations

Table 5.02: Cardinality of relations

Relation	Cardinality	Notes
users to sessions	1 : m	A user logs in many times over their life; the application keeps only one ACTIVE at a time but retains history.
users to carts	1 : 1	Enforced by the unique foreign key.
carts to cart_items	1 : m	Composition: items are deleted with the cart (cascade, orphanRemoval).
cart_items to products	m : 1	Many carts may contain the same product.
categories to products	1 : m	A product belongs to exactly one category.
users to orders	1 : m	
orders to order_items	1 : m	Composition, as for carts.
order_items to products	m : 1	The foreign key is kept even after soft deletion of the product, which is why deletion is soft.
orders to payments	1 : m	Failed attempts are retained for audit; at most one is SUCCESS.
products to carts/orders	m : m (via item tables)	The many-to-many between products and carts, and between products and orders, is resolved through cart_items and order_items which carry the quantity attribute.

5.5 Design decisions

5.5.1 Soft deletion

Products are never physically deleted. `DELETE /products/{id}` sets `deleted = true`, and every read query includes `deleted = false`. The reason is referential: `order_items.product_id` must remain valid so that order history can still display what was bought. A hard delete would either fail on the foreign key or require cascading the deletion into order history, both unacceptable. Soft deletion also allows an administrator to restore a product withdrawn by mistake.

5.5.2 Price snapshot on order_items

`order_items.unit_price` duplicates `products.price` at the moment of purchase. This is deliberate denormalisation. Without it, an administrator changing a price would retroactively change the total of every historical order, and a customer's invoice would no longer match what they paid. The small storage cost buys a strong correctness guarantee.

5.5.3 Enumerations as strings

All enums are persisted with `EnumType.STRING` rather than ordinal. Ordinals break silently when a value is inserted in the middle of an enum; strings are self-describing in the database and safe to reorder in code.

5.5.4 Surrogate keys and timestamps everywhere

Every table has an auto-increment `id`, even join-like tables such as `cart_items`. This keeps the JPA mapping uniform (no composite-key classes) and gives every row a stable identity for logging and

debugging. The timestamps support ordering (orders newest first) and future analytics without schema changes.

5.5.5 Why MySQL and not a document store

The data is strongly relational and the critical operation, order placement, needs multi-row atomicity: decrement several product rows, insert an order and its lines, delete cart lines, all or nothing. A relational database with ACID transactions is the natural fit. Document stores excel at flexible schemas and horizontal write scaling, neither of which is the bottleneck here.

5.6 Cache layout

Redis is not a relational store but it is part of the data design. Table 5.03 shows the two keys used by the popular-products cache. The split into a HASH (data) and a SET (membership) means a read is exactly two commands regardless of how many products are popular, and an eviction is two constant-time operations.

Table 5.03: Redis key layout for the popular-products cache

Key	Type	Content	TTL
popular_products	HASH	field = product id as string; value = JSON of the Product entity (with an `@class` property so it deserialises back to Product).	600 s, refreshed on every write
popular_products:ids	SET	Product ids as strings. Enumerated with SMEMBERS to drive the HMGET.	600 s, refreshed on every write

Both keys expire together so the cache is either whole or absent; a partial state (ids without data) is tolerated by the read path, which treats it as a miss.

Chapter 6: Feature Development Process

6.1 Feature selected: popular products with Redis caching

The feature chosen for detailed treatment is `GET /products/popular`, which returns the products most likely to be viewed on a storefront's landing page: those whose titles contain a configurable set of keywords such as "iphone", "macbook", "jordan" and "rolex". It was chosen because it exercises the full MVC path, involves a measurable performance optimisation, and raises real operational questions (what happens when the cache is stale, or down) whose answers shape the design.

The business motivation is simple. On a retail site a small fraction of products receives a large fraction of views. Serving that fraction from memory relieves the database of its most repetitive work, lowers tail latency for the page customers see first, and leaves database capacity for the queries that genuinely need it (search, cart, order). The same technique applies to "trending", "deals of the day" or "recently viewed" widgets.

6.2 Request flow

6.2.1 API request and response payload

The endpoint is public and takes no parameters:

```
GET /products/popular HTTP/1.1
Host: localhost:8080
Accept: application/json
```

The response is a JSON array of products. The example below is abridged from a real run of the application against the seeded catalogue:

```
HTTP/1.1 200 OK
Content-Type: application/json

[
  {
    "id": 1,
    "createdAt": "2026-10-06T16:02:34.492354Z",
    "updatedAt": "2026-10-06T16:02:34.492358Z",
    "deleted": false,
    "title": "iPhone 15 Pro",
    "description": "6.1-inch Super Retina XDR, A17 Pro chip, 256 GB",
    "price": 134900.0,
    "imageUrl": "https://picsum.photos/seed/iphone15pro/400",
    "stock": 25,
    "category": { "id": 1, "name": "Electronics", "description": null, ... },
    "externalId": null
  },
  { "id": 2, "title": "MacBook Air M3", "price": 144900.0, ... },
  { "id": 4, "title": "Air Jordan 1 Retro High", "price": 16995.0, ... },
  { "id": 6, "title": "Rolex Submariner", "price": 985000.0, ... }
]
```

6.2.2 Controller

`ProductController` receives the request. Like every controller in the project it contains no logic beyond delegation:

```
@GetMapping("/popular")
public ResponseEntity<List<Product>> getPopular() {
    return ResponseEntity.ok(popularProductCache.getPopularProducts());
}
```

6.2.3 Service

`PopularProductCacheService` is the service that picks up the request. Its read method implements cache-aside with a database fallback:

```
public List<Product> getPopularProducts() {
    try {
        Set<Object> ids = redisTemplate.opsForSet().members(idSetKey);
        if (ids != null && !ids.isEmpty()) {
            List<Object> hashKeys = new ArrayList<>();
            ids.forEach(id -> hashKeys.add(id.toString()));
            List<Object> cached = redisTemplate.opsForHash().multiGet(hashKey,
hashKeys);
            List<Product> products = new ArrayList<>();
            for (Object o : cached) {
                if (o instanceof Product p) {
                    products.add(p);
                }
            }
            if (!products.isEmpty()) {
                return products;                        // cache hit
            }
        }
        List<Product> fromDb = loadPopularFromDb();    // cache miss
        warm(fromDb);
        return fromDb;
    } catch (RuntimeException e) {
        log.warn("Redis unavailable, serving popular products from DB: {}",
e.getMessage());
        return loadPopularFromDb();                    // graceful degradation
    }
}
```

Three paths exist. On a hit, two Redis commands return the data and the database is not touched. On a miss, the service runs one JPQL query per keyword (`findByTitleKeyword`, a case-insensitive LIKE), de-duplicates with a `LinkedHashSet`, writes the result back to Redis and sets the TTL. On any Redis exception the service logs at WARN and returns the database result; the customer never sees the outage.

6.2.4 Repository

The fallback query is a JPQL method on `ProductRepository`:

```
@Query("SELECT p FROM Product p WHERE p.deleted = false "
     + "AND LOWER(p.title) LIKE LOWER(CONCAT('%', :keyword, '%'))")
List<Product> findByTitleKeyword(@Param("keyword") String keyword);
```

The LIKE with a leading wildcard is a full scan of the title index. For a catalogue of thousands of rows this is a few milliseconds; for millions it would be the reason the cache exists.

6.2.5 Write path: keeping the cache coherent

A cache that can serve a stale price is worse than no cache. `SelfProductService` therefore calls the cache after every successful write, inside the same transactional method, so the cache changes immediately rather than at TTL expiry:

```
public void onProductChanged(Product product) {
    try {
        String id = String.valueOf(product.getId());
        if (isPopular(product) && !product.isDeleted()) {
            redisTemplate.opsForHash().put(hashKey, id, product);
            redisTemplate.opsForSet().add(idSetKey, id);
            refreshTtl();
        } else {
            evict(id);          // renamed away from a keyword, or soft-deleted
        }
    } catch (RuntimeException e) {
        log.warn("Redis unavailable, skipping cache update for product {}",
product.getId());
    }
}
```

Figure 6.01 shows both paths side by side, and Table 6.01 enumerates the Redis commands each one issues.

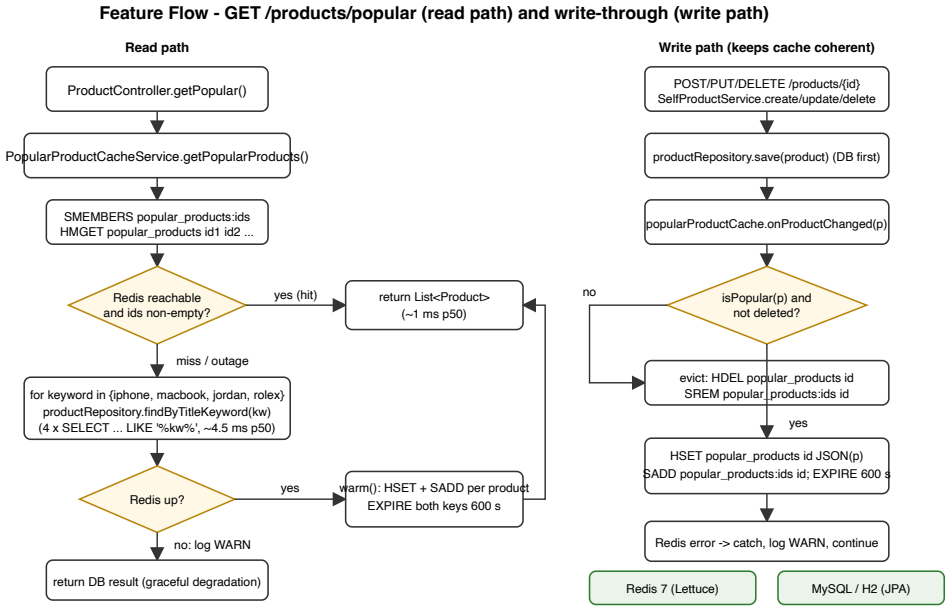


Figure 6.01: Popular-products feature flow: read path and write-through

Table 6.01: Redis commands issued per request path

Path	Commands	Complexity
Read, cache hit	SMEMBERS popular_products:ids; HMGET popular_products id1 ... idk	O(k), 2 round trips
Read, cache miss	SMEMBERS; then per product HSET + SADD; then EXPIRE x 2	O(k), 2k + 3 round trips (once per TTL)
Read, Redis down	SMEMBERS fails fast (2 s timeout); no further Redis calls	database only
Write, product popular	HSET; SADD; EXPIRE x 2	O(1), 4 round trips
Write, product not popular or deleted	HDEL; SREM	O(1), 2 round trips

6.3 Development history and a defect found by testing

The service was first written against a mocked `RedisTemplate` and its five unit tests (Table 6.03) passed. The first end-to-end smoke test against a real Redis instance, however, showed `redis-cli KEYS '*'` returning nothing after a request that had returned the correct products. The application log explained it:

```
WARN c.s.e.s.PopularProductCacheService : Redis unavailable, serving popular products
from DB:
Could not write JSON: Java 8 date/time type `java.time.Instant` not supported by
default:
add Module "com.fasterxml.jackson.datatype:jackson-datatype-jsr310" ...
```

The default `GenericJackson2JsonRedisSerializer` builds its own `ObjectMapper` without the JSR-310 module, so serialising the `createdAt` field failed, the exception was caught by the fallback, and every request silently went to the database. The fallback did exactly what it was designed to do, which is why the bug was invisible from the API. The fix in `RedisConfig` supplies a purpose-built mapper:

```
private static ObjectMapper redisObjectMapper() {
    ObjectMapper mapper = new ObjectMapper();
    mapper.registerModule(new JavaTimeModule());
    mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
    mapper.activateDefaultTyping(
            BasicPolymorphicTypeValidator.builder()
                    .allowIfSubType("com.shivendra.ecommerce.models")
                    .allowIfSubType("java.util")
                    .allowIfSubType("java.time")
                    .build(),
            ObjectMapper.DefaultTyping.NON_FINAL,
            JsonTypeInfo.As.PROPERTY);
    return mapper;
}
```

Default typing writes an `@class` property into each cached value so it deserialises back to `Product` rather than `LinkedHashMap`; the validator restricts which classes may be instantiated, closing the deserialisation-gadget attack that unrestricted default typing is known for. After the fix, `redis-cli` showed the expected keys and the MONITOR output for a request was exactly the two commands predicted in Table 6.01.

Two lessons were recorded. First, graceful degradation must be paired with observability, otherwise it hides defects; the WARN log line was the only signal. Second, a mocked integration test cannot catch serialisation problems; at least one test must talk to the real dependency.

6.4 Performance optimisation and benchmark

6.4.1 Method

The benchmark script `scripts/bench_popular_cache.py` in the repository measures `GET /products/popular` on a single JVM in three configurations. It first issues fifty warm-up requests so the JIT compiler and connection pools are steady. It then measures two hundred requests with the Redis keys deleted before every call (the database path), two hundred requests with the cache warm (the Redis path), and two hundred requests to `GET /search?q=iphone` as a reference for a single LIKE query. Latency is wall-clock time of the HTTP round trip measured from the client with `time.perf_counter()`. The environment was a laptop with Apple Silicon, H2 in-memory as the database and Redis 7 on localhost; absolute numbers are therefore small, and the comparison between paths is the meaningful result.

6.4.2 Results

Table 6.02: Benchmark: latency of GET /products/popular, database vs Redis

Path (200 requests)	p50 (ms)	p95 (ms)	p99 (ms)	mean (ms)
Database: cache flushed before each request (4 LIKE queries + warm)	4.52	6.55	7.20	4.77
Redis: cache warm (SMEMBERS + HMGET)	0.99	1.87	2.32	1.12
Reference: GET /search?q=iphone (1 LIKE query)	1.24	1.89	2.44	1.37
Improvement, database to Redis	**4.6x**	**3.5x**	**3.1x**	**4.3x**

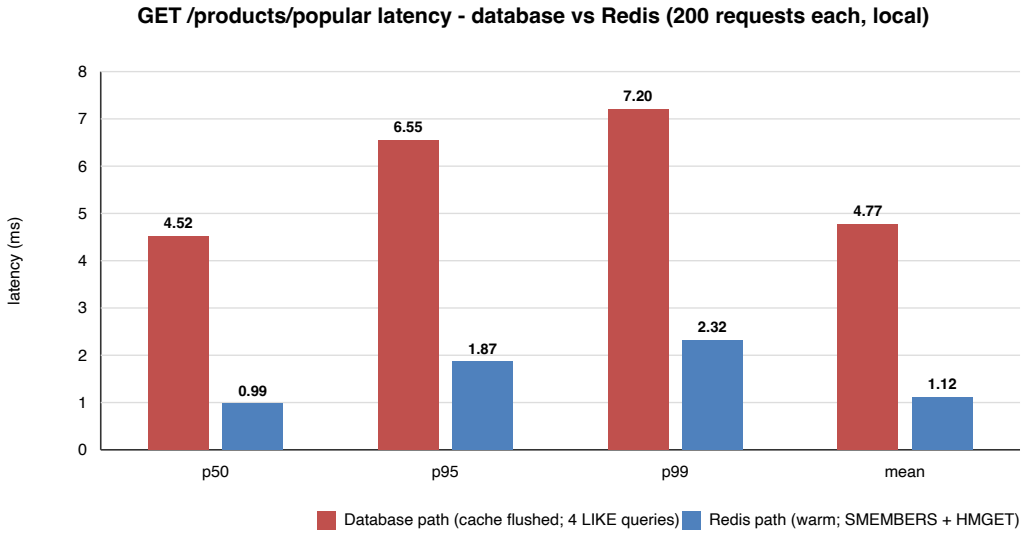


Figure 6.02: Benchmark chart: database vs Redis latency

6.4.3 Interpretation

The Redis path cuts median latency by about 3.5 ms and 95th-percentile latency by about 4.7 ms. Note that the database path includes the cost of warming the cache after the miss, which slightly inflates it; even excluding that, four LIKE scans will always cost more than two O(k) Redis commands. The reference row is instructive: a single LIKE query on this tiny catalogue (ten rows) is already comparable to the Redis path, which shows that the cache's advantage grows with catalogue size and keyword count, not with request volume alone. On a production catalogue of one million products a leading-wildcard LIKE would take tens to hundreds of milliseconds per keyword, while the Redis path would remain around one millisecond.

Equally important is what the cache removes from the database: on a warm cache the popular endpoint issues zero SQL statements. For a landing-page widget called on every visit, that is the difference between the database serving carts and orders comfortably and the database saturating on repetitive reads.

6.4.4 Trade-offs

- **Staleness window.** Write-through means changes made through this application are visible immediately. Changes made directly in the database (for example by a bulk import script) are visible only after the 600-second TTL. The TTL is the bound on staleness in that case.

- **Memory.** Each cached product is roughly 500 bytes of JSON including the `@class` markers; even a thousand popular products would use under a megabyte.
- **Keyword definition.** Popularity is defined by configuration rather than by measured views. A production system would derive it from an analytics pipeline; the cache layer would be unchanged.
- **Partial failure.** If Redis fails between HSET and SADD on a write, the data and the id set can disagree briefly; the read path tolerates missing data for an id (it simply omits it) and the next TTL expiry resets both keys.

6.5 Tests

The feature has five unit tests in `PopularProductCacheServiceTest`, listed in Table 6.03, plus the manual smoke test described above. The tests mock `RedisTemplate`, `HashOperations` and `SetOperations` so they run without Redis.

Table 6.03: Unit tests covering the popular-products cache

Test	Asserts
cacheHitSkipsDatabase	When the id set and hash return data, the repository is never called.
cacheMissLoadsFromDbAndWarms	Empty set leads to repository calls per keyword, HSET and SADD per product, and EXPIRE with the configured TTL.
redisOutageFallsBackToDb	A `RedisConnectionFailureException` on SMEMBERS still yields the database result with no exception propagated.
onProductChangedWritesPopularAndEvictsOthers	A product with a keyword title is HSET and SADDed; one without is HDELed and SREMed.
isPopularMatchesKeywordsCaseInsensitively	"Apple IPHONE 15" is popular; "Wooden chair" is not.

Chapter 7: Deployment Flow

7.1 Overview

The application is packaged as a single executable jar and is designed to run as several identical instances behind a load balancer, with all state in MySQL and Redis. This chapter describes the target production topology on Amazon Web Services shown in Figure 7.01, the role of each service, the network and security configuration, and the deployment procedure. The region chosen is ap-south-1 (Mumbai) for proximity to Indian customers and payment providers.

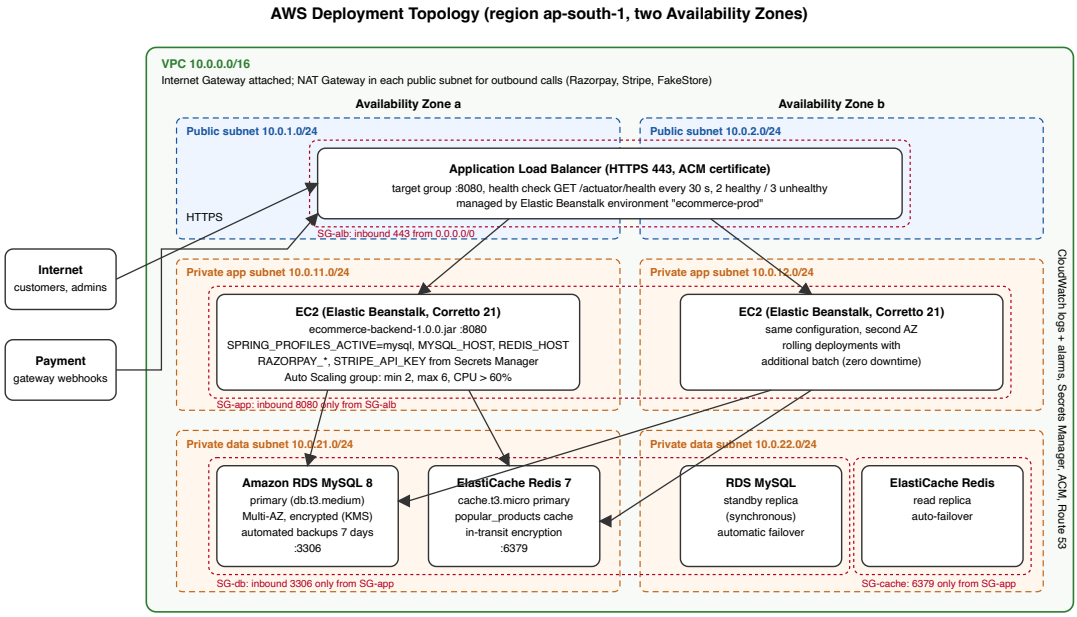


Figure 7.01: AWS deployment topology

7.2 Virtual Private Cloud (VPC)

All resources live in one VPC with the CIDR block 10.0.0.0/16, spread across two Availability Zones so that the loss of one data centre does not take the service down. Subnets are divided into three tiers (Table 7.01). Only the public tier has a route to the Internet Gateway; the application and data tiers route outbound traffic through a NAT Gateway in each public subnet, which the application needs to reach Razorpay, Stripe and FakeStore. Nothing in the private tiers has a public IP address.

Table 7.01: VPC subnet plan

Tier	AZ a	AZ b	Contents	Routing
Public	10.0.1.0/24	10.0.2.0/24	Application Load Balancer nodes, NAT Gateways	0.0.0.0/0 via Internet Gateway
Private application	10.0.11.0/24	10.0.12.0/24	EC2 instances managed by Elastic Beanstalk	0.0.0.0/0 via NAT Gateway
Private data	10.0.21.0/24	10.0.22.0/24	RDS MySQL primary and standby; ElastiCache Redis primary and replica	no internet route

7.3 Elastic Compute Cloud (EC2)

The application runs on EC2 instances using the Elastic Beanstalk "Corretto 21" platform, which provides Amazon's OpenJDK distribution, an nginx reverse proxy and the Beanstalk agent. The instance type t3.small (2 vCPU, 2 GB) is sufficient for the JVM with a 1 GB heap at the expected load; the Auto Scaling group keeps a minimum of two instances, one per Availability Zone, and scales up to six when average CPU exceeds 60 percent for five minutes. Instances are stateless: a restart loses nothing because sessions are in MySQL and the cache is in Redis. Instance profiles grant the IAM permissions needed to read secrets and write CloudWatch logs; no credentials are placed on disk.

7.4 Security Groups

Security groups are stateful firewalls attached to each resource. The rules follow the principle of least privilege: each tier accepts traffic only from the security group of the tier in front of it, never from CIDR ranges (Table 7.02). This means that even if an attacker obtained a foothold in the VPC they could not connect to the database without first compromising an application instance.

Table 7.02: Security group rules

Group	Attached to	Inbound	Outbound
SG-alb	Application Load Balancer	TCP 443 from 0.0.0.0/0 (TCP 80 redirects to 443)	TCP 8080 to SG-app
SG-app	EC2 instances	TCP 8080 from SG-alb only	TCP 3306 to SG-db; TCP 6379 to SG-cache; TCP 443 to 0.0.0.0/0 (payment gateways)
SG-db	RDS MySQL	TCP 3306 from SG-app only	none required
SG-cache	ElastiCache Redis	TCP 6379 from SG-app only	none required

Administrative access to instances uses AWS Systems Manager Session Manager rather than SSH, so port 22 is not open anywhere.

7.5 Relational Database Service (RDS)

MySQL 8 runs on Amazon RDS with Multi-AZ deployment: a primary instance in one Availability Zone and a synchronously replicated standby in the other, with automatic failover in about a minute if the primary fails. The instance class db.t3.medium (2 vCPU, 4 GB) is a reasonable start; storage is gp3 with encryption at rest through KMS; automated backups are retained for seven days with point-in-time recovery. The application connects through the RDS endpoint hostname, which stays constant across failover, using the `mysql` Spring profile. Connection pooling is provided by HikariCP (the Spring Boot default) with a pool size of ten per instance, so six instances hold at most sixty connections, well within the limit of the instance class. RDS Performance Insights is enabled to watch for slow queries, which is how the indexes in Chapter 5 would be validated against real traffic.

7.6 Cache (ElastiCache for Redis)

Redis 7 runs on Amazon ElastiCache as a replication group with one primary and one read replica across the two zones, with automatic failover and in-transit encryption enabled. The node type cache.t3.micro (0.5 GB) is ample for the popular-products cache, which uses well under a megabyte. The application

receives the primary endpoint through the `REDIS_HOST` environment variable. Because Chapter 6 designed the cache as an accelerator rather than a dependency, and because the Redis health indicator is disabled in the application, a cache outage shows up as a latency increase and WARN logs rather than as failed health checks or removed instances.

7.7 Managed infrastructure: Elastic Beanstalk

Elastic Beanstalk is the orchestration layer. It provisions and owns the load balancer, the Auto Scaling group, the launch template, the CloudWatch alarms and the deployment process, so the team manages an application rather than infrastructure. The environment is configured as follows:

- **Platform:** Corretto 21 on Amazon Linux 2023. The jar is uploaded as the application version.
- **Load balancer:** Application Load Balancer, HTTPS listener with an ACM certificate, target group on port 8080, health check `GET /actuator/health` every 30 seconds, two consecutive successes to mark healthy, three failures to mark unhealthy.
- **Deployment policy:** rolling with an additional batch. Beanstalk launches one extra instance, deploys to it, waits for it to pass health checks, then rolls the remaining instances one at a time. Capacity never drops below the configured minimum and a failed health check halts the rollout.
- **Configuration:** environment properties (Table 7.03) are injected as environment variables that Spring Boot binds to configuration keys. Secrets are referenced from AWS Secrets Manager and resolved at instance start.
- **Logging:** application logs stream to CloudWatch Logs; alarms fire on 5xx rate above 1 percent and on p95 latency above 500 ms.

Table 7.03: Environment properties injected by Elastic Beanstalk

Variable	Value / source	Bound to
SPRING_PROFILES_ACTIVE	mysql	Activates application-mysql.properties
MYSQL_HOST, MYSQL_PORT, MYSQL_DB	RDS endpoint, 3306, ecommerce	spring.datasource.url
MYSQL_USER, MYSQL_PASSWORD	Secrets Manager	spring.datasource.username / password
REDIS_HOST, REDIS_PORT	ElastiCache primary endpoint, 6379	spring.data.redis.host / port
RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET	Secrets Manager	payment.razorpay.key-id / key-secret
STRIPE_API_KEY	Secrets Manager	payment.stripe.api-key
SEED_ENABLED	false	seed.enabled (no demo data in production)

7.8 Deployment procedure

The procedure from a developer's machine, using the Elastic Beanstalk command line interface, is:

```
# 1. build and test
./mvnw clean package

# 2. first time only: create the application and environment
eb init ecommerce-backend --platform "Corretto 21" --region ap-south-1
eb create ecommerce-prod --instance_type t3.small --min-instances 2 --max-instances 6
\
   --vpc.id vpc-xxxx --vpc.ec2subnets subnet-app-a,subnet-app-b \
   --vpc.elbsubnets subnet-pub-a,subnet-pub-b --vpc.elbpublic \
   --envvars SPRING_PROFILES_ACTIVE=mysql,SEED_ENABLED=false

# 3. every release
eb deploy ecommerce-prod
eb health ecommerce-prod
```

The payment gateways are then configured in their dashboards to deliver webhooks to `https://api.example.com/payments/webhook`. In a team setting these commands would run from a CI pipeline (GitHub Actions or AWS CodePipeline) on every merge to the main branch, after the test suite passes, and the `eb create` step would be replaced by infrastructure as code (CloudFormation or Terraform) so the VPC, security groups, RDS and ElastiCache are reproducible.

7.9 Indicative cost

Table 7.04 gives an order-of-magnitude monthly cost for the topology at on-demand prices in ap-south-1. The figures are indicative and would be reduced with reserved instances or by starting with single-AZ RDS and ElastiCache in a pre-production stage.

Table 7.04: Indicative monthly cost of the production topology

Resource	Configuration	Approx. USD / month
EC2	2 x t3.small, 730 h	35
Application Load Balancer	1 ALB, low LCU usage	25
NAT Gateway	2 x gateway hours plus modest data	70
RDS MySQL	db.t3.medium Multi-AZ, 50 GB gp3	140
ElastiCache Redis	2 x cache.t3.micro	30
Secrets Manager, CloudWatch, data transfer	small	15
Total		**about 315**

The two largest items, Multi-AZ RDS and the NAT Gateways, are the price of availability and of keeping the application tier off the public internet. Both are defensible for a store that processes payments.

Chapter 8: Technologies Used

8.1 Summary

Table 8.01: Technology stack summary

Concern	Choice	Role in this project
Language	Java 21 (Amazon Corretto)	All application code; records, pattern matching for instanceof, text blocks.
Framework	Spring Boot 3.3.5	Dependency injection, embedded Tomcat, auto-configuration, Actuator health.
Web	Spring Web MVC, Bean Validation	REST controllers, request validation, interceptors.
Persistence	Spring Data JPA, Hibernate 6	Entity mapping, repositories, JPQL, transactions.
Database	MySQL 8 (production), H2 2.x (dev/test)	Nine relational tables.
Cache	Redis 7 via Spring Data Redis and Lettuce	Popular-products HASH and SET.
Security	spring-security-crypto (BCrypt)	Password hashing; custom session tokens.
Payments	razorpay-java 1.4.6, stripe-java 26.0.0	Payment Links and Checkout Sessions.
Build	Maven 3.9 (wrapper)	Dependency management, test execution, executable jar.
Testing	JUnit 5, Mockito, AssertJ, Spring Boot Test	30 tests.
Infrastructure	Docker Compose (local), AWS (production)	MySQL and Redis containers; VPC, Beanstalk, RDS, ElastiCache.

8.2 Java 21

Description. Java is a statically typed, garbage-collected language running on the Java Virtual Machine. Version 21 is a long-term-support release and brings records (immutable data carriers used here for `LoginResponseDto`, `PaymentResponseDto` and `PaymentLinkResult`), pattern matching for `instanceof` (used when reading cached objects back from Redis), sealed types, and virtual threads. The JVM's just-in-time compiler produces performance close to native code for long-running services, and its mature garbage collectors (G1 by default) keep pause times low. The project uses Amazon Corretto, a free production-ready distribution of OpenJDK.

Real-life use. Java is the dominant language for large-scale backend systems in finance, retail and telecommunications because of its performance, tooling and the size of its ecosystem. It is particularly suited to services that must run for months without restart and be maintained by large teams.

Examples. Amazon's retail website and much of AWS run on Java services; Netflix's backend microservices are predominantly Java on Spring Boot; LinkedIn, Uber and Alibaba run core platforms on the JVM.

8.3 Spring Boot

Description. Spring Boot is an opinionated layer over the Spring Framework that removes configuration boilerplate. Its auto-configuration inspects the classpath and configures beans automatically: adding `spring-boot-starter-data-redis` is enough to obtain a configured `RedisConnectionFactory`; adding `spring-boot-starter-web` starts an embedded Tomcat. The core idea of Spring, dependency injection, is what makes the project's layering enforceable: every service declares its collaborators as constructor parameters, Spring supplies them at runtime, and tests supply mocks. Boot also provides profiles (used here to switch H2, MySQL and test configurations), externalised configuration (every property can be overridden by an environment variable, the basis of the AWS deployment), and Actuator for health and metrics endpoints.

Real-life use. Spring Boot is the default choice for Java microservices and monoliths alike. It shortens the path from an empty directory to a running HTTP service to minutes, and its conventions make codebases recognisable to any Java developer.

Examples. Netflix built its Spring Cloud ecosystem on Boot; Walmart, Capital One, Zomato and Flipkart run large fleets of Spring Boot services.

8.4 Spring Data JPA and Hibernate

Description. The Java Persistence API is a specification for mapping Java objects to relational tables; Hibernate is its most widely used implementation. Spring Data JPA adds repository interfaces whose methods are implemented from their names (`findByEmailIgnoreCase`, `findByIdAndDeletedFalse`) or from `@Query` annotations, removing almost all hand-written data-access code. Hibernate manages the identity of loaded entities within a transaction, generates SQL for the target dialect, and handles relationships, cascades and lazy loading. The project relies on it for cascading cart and order items, for `orphanRemoval` to delete removed lines, for automatic timestamps, and for the transaction demarcation that makes order placement atomic.

Real-life use. Object-relational mapping is used wherever a domain model is richer than flat rows: e-commerce, banking, enterprise resource planning. It speeds development and reduces SQL injection risk through parameterised queries, at the cost of needing care with lazy loading and N+1 query patterns.

Examples. Most Java enterprise applications, including those at banks such as HDFC and ICICI, SAP Hybris commerce, and the many Spring Boot services mentioned above.

8.5 MySQL

Description. MySQL is an open-source relational database management system with ACID transactions through the InnoDB storage engine, row-level locking, B-tree secondary indexes, replication, and a very large operational knowledge base. The project depends on InnoDB's transactional guarantees for the order placement flow and on secondary indexes for the catalogue queries. H2 is used locally in MySQL compatibility mode so that developers and the test suite need no database installation; the switch to MySQL is a Spring profile.

Real-life use. MySQL is the most widely deployed open-source relational database. It underpins content management, e-commerce, software-as-a-service products and analytics staging. Managed offerings such as Amazon RDS and Aurora remove most of the operational burden.

Examples. Facebook (one of the largest MySQL deployments in the world), YouTube, Shopify, Uber (via its Schemaless layer), Booking.com and Zomato.

8.6 Redis

Description. Redis is an in-memory data structure server. Beyond simple key-value strings it offers hashes, sets, sorted sets, lists, streams and geospatial indexes, all operated on by atomic commands with sub-millisecond latency. Data lives in memory with optional persistence to disk. The project uses a HASH to store product JSON by id and a SET for membership, with TTLs for expiry; Spring Data Redis provides `RedisTemplate` with pluggable serialisers, and Lettuce is the underlying non-blocking client. Chapter 6 shows the measured effect: a four-fold latency reduction and the removal of all SQL from the hot endpoint.

Real-life use. Redis is used as a cache in front of databases, as a session store for web applications, for rate limiting and leaderboards (sorted sets), for pub/sub messaging, and for distributed locks. Its single-threaded command execution gives atomicity without locks, which is why counters and queues built on it are reliable.

Examples. Twitter's timeline cache, GitHub's job queues, Stack Overflow's page cache, Snapchat's story metadata, and almost every large e-commerce site's product and session cache.

8.7 Spring Security Crypto and session-based authentication

Description. `spring-security-crypto` is the standalone cryptography module of Spring Security. The project uses its `BCryptPasswordEncoder`, which applies the bcrypt adaptive hash with a random salt and a configurable work factor (ten rounds by default, roughly 100 ms per hash), making brute-force attacks on a leaked database impractical. Authentication itself is a deliberately small custom mechanism: 32 bytes from `SecureRandom`, encoded as URL-safe base64, stored in a `sessions` row with an expiry and status. A `HandlerInterceptor` validates the token on protected routes. This was chosen over JSON Web Tokens because a single backend gains little from stateless tokens and loses instant revocation; over full Spring Security because the configuration surface of that framework would have obscured the simple requirement.

Real-life use. Password hashing with bcrypt, scrypt or Argon2 is the baseline for any system storing credentials. Server-side session stores are standard for web applications that need logout and session listing; stateless JWTs are preferred when many independent services must validate tokens without a shared store.

Examples. Django, Ruby on Rails and Laravel all ship bcrypt-based password hashing and database or cache-backed sessions by default; large sites such as GitHub use server-side sessions for exactly the revocability reason.

8.8 Razorpay and Stripe

Description. Both are payment service providers that let a merchant accept cards, UPI, net banking and wallets without handling card data directly, which keeps the merchant out of most of the PCI DSS compliance scope. Razorpay, an Indian provider, offers Payment Links: the merchant creates a link for an amount and customer, the customer pays on Razorpay's hosted page, and Razorpay notifies the merchant by webhook. Stripe, a global provider, offers Checkout Sessions with the same shape. The project integrates both through their official Java SDKs behind the `PaymentGateway` strategy, sending amounts

in the smallest currency unit (paise), passing the order id as the reference, and reconciling the result through an idempotent webhook endpoint.

Real-life use. Hosted checkout pages are the fastest and safest way to accept payments for small and medium merchants. Supporting two providers gives resilience against provider outages and lets a merchant route by cost, geography or payment method.

Examples. Razorpay processes payments for Swiggy, Zomato, Cred and IRCTC; Stripe for Amazon (in some regions), Shopify, Zoom and Lyft.

8.9 Maven and the project's dependencies

Description. Apache Maven is the build and dependency-management tool. The project includes the Maven wrapper (`mvnw`) so a fresh machine needs only a JDK. The Spring Boot parent POM manages dependency versions so that the libraries in Table 8.02 are known to work together. `./mvnw test` compiles and runs the test suite; `./mvnw package` additionally produces the executable jar with all dependencies embedded.

Table 8.02: Maven dependencies and their purpose

Dependency	Purpose
spring-boot-starter-web	Spring MVC, Jackson JSON, embedded Tomcat
spring-boot-starter-validation	Bean Validation (@NotBlank, @Email, @Min) on DTOs
spring-boot-starter-actuator	/actuator/health for the load balancer
spring-boot-starter-data-jpa	Spring Data repositories, Hibernate, HikariCP connection pool
mysql-connector-j (runtime)	MySQL JDBC driver for the mysql profile
h2 (runtime)	In-memory database for the default and test profiles
spring-boot-starter-data-redis	RedisTemplate, Lettuce client
spring-security-crypto	BCryptPasswordEncoder
razorpay-java 1.4.6	Razorpay Payment Links API client
stripe-java 26.0.0	Stripe Checkout Sessions API client
lombok (optional)	Generates getters and setters on entities and DTOs
spring-boot-starter-test	JUnit 5, Mockito, AssertJ, Spring test support

8.10 Testing: JUnit 5, Mockito and AssertJ

Description. JUnit 5 is the test runner; Mockito creates mock implementations of repositories and gateways so that each service is tested in isolation; AssertJ supplies fluent, readable assertions. One Spring Boot test loads the full application context against H2 with a `test` profile to catch wiring errors. The 30 tests cover password hashing and session expiry, cart merging and stock guards, transactional order placement and cancellation, search validation, the three cache paths and write-through, and payment routing and webhook idempotency. They run in about ten seconds with no external services.

Real-life use. Fast, isolated unit tests are what allow a team to change code confidently and to run the suite on every commit in continuous integration. Mocking at the repository boundary tests the business rules without the cost of a database.

Examples. Virtually every Java project; Spring itself, Elasticsearch and Kafka use JUnit and Mockito extensively.

8.11 Docker Compose and AWS

Description. Docker Compose starts MySQL 8.4 and Redis 7 locally with one command for developers who want to exercise the production profile. AWS provides the production platform described in Chapter 7: VPC for networking, EC2 for compute, Elastic Beanstalk for orchestration and deployment, RDS for managed MySQL, ElastiCache for managed Redis, and supporting services (Secrets Manager, CloudWatch, Certificate Manager).

Real-life use. Containers make local environments reproducible; managed cloud services let small teams run highly available databases and caches without database administrators. The combination of a stateless application tier and managed stateful services is the standard shape of a modern web backend.

Examples. Netflix, Airbnb and Zerodha run on AWS; Elastic Beanstalk is used by companies such as Zillow and Samsung Business Services for Java workloads.

Chapter 9: Conclusion

9.1 Summary of work

This project set out to build a complete, operable backend for an online store and to document it to a professional standard. The delivered system exposes 27 REST endpoints across authentication, catalogue, search, cart, order and payment modules; persists nine relational tables with carefully chosen indexes; accelerates its hottest read with a Redis cache that degrades gracefully; integrates two independent payment providers behind one interface; and is verified by thirty automated tests and a reproducible benchmark. A full AWS deployment design with network isolation, least-privilege security groups and managed data services completes the picture. Every objective in Table 2.01 was met.

9.2 Key takeaways

- **Transactions are the primitive that makes inventory correct.** Order placement validates and decrements stock for every line inside one database transaction. The unit test that forces an `InsufficientStockException` on the second line and verifies the first line's stock is untouched captures the whole point of ACID in one assertion.
- **Snapshot what must not change.** Copying the unit price into the order line is a few bytes of denormalisation that prevents an entire class of financial inconsistency.
- **Design for the dependency being down.** Treating Redis as an accelerator, wrapping every call, and excluding it from the health check means a cache outage is a latency event, not an availability event. The flip side, learned the hard way, is that silent fallback needs loud logging or defects hide behind it.
- **Integration tests find what mocks cannot.** The Jackson JSR-310 serialisation defect passed every unit test and was found only when real Redis was involved. One honest end-to-end test per external system is worth many mocked ones.
- **Interfaces at the seams buy flexibility cheaply.** `ProductService` and `PaymentGateway` each cost a few lines and let the system swap its catalogue source and its payment provider by configuration.
- **Idempotency is a webhook's first requirement.** Payment providers retry deliveries; the handler that checks the current state before changing it is what prevents an order from being processed twice.
- **Measure, then claim.** The benchmark turned "Redis is faster" into "p95 fell from 6.55 ms to 1.87 ms on this hardware with this catalogue", which is a statement a reviewer can reproduce and challenge.

9.3 Practical applications

Although framed as a store, the system is a template for any transactional platform. Session-token authentication with revocation serves any web application. Cart-to-order with stock reservation is seat booking in a cinema, inventory allocation in a warehouse, or slot reservation for a service appointment. The payment strategy and webhook reconciliation apply unchanged to subscriptions, donations or marketplace payouts. The popular-products cache is the pattern behind trending lists, personalised home pages and configuration lookups in countless services. The AWS topology is the standard shape of a three-tier web application and would host most of these variants without change.

For the author, the project also serves as a reference for professional work: the trade-off notes in Chapters 4 to 6 are the kind of reasoning expected in design reviews, and the operational concerns in Chapter 7 are those an on-call engineer must understand.

9.4 Limitations and suggested improvements

Table 9.01: Limitations and suggested improvements

Limitation	Impact	Suggested improvement
Webhook signature is not verified	An attacker who learns a provider reference id could mark an order paid.	Verify the `X-Razorpay-Signature` HMAC and Stripe's `Stripe-Signature` header using the webhook secret before processing; both SDKs provide helpers.
Catalogue write endpoints are unauthenticated	Anyone can create or delete products in this build.	Extend `AuthInterceptor` to enforce the ADMIN role on POST/PUT/DELETE under /products and /categories; the `Role` enum already exists.
Stock decrement is not protected against concurrent orders at the row level	Two simultaneous orders for the last unit could both pass the check under READ COMMITTED.	Use `SELECT ... FOR UPDATE` via `@Lock(PESSIMISTIC_WRITE)` on the product load, or an optimistic `@Version` column with retry.
Schema managed by Hibernate ddl-auto	Unsafe for production: no review of generated DDL, no rollback path.	Introduce Flyway migrations and set ddl-auto to validate.
Popularity is configured, not measured	The cache serves a fixed keyword list rather than actual demand.	Count views in a Redis sorted set and rebuild the popular set from the top-N periodically.
Search uses LIKE with a leading wildcard	Full index scan; fine for thousands of products, slow for millions.	MySQL FULLTEXT index, or an external search engine such as Elasticsearch or OpenSearch.
Monetary values are DOUBLE	Floating-point rounding can accumulate in totals.	Store amounts as BIGINT paise or DECIMAL(12,2) and use `BigDecimal` in code.
Error messages expose internal detail on 500	Leaks exception text to clients.	Return a correlation id and log the detail server-side only.
No rate limiting on login	Credential stuffing is possible.	Per-IP and per-account counters in Redis with exponential back-off.
Cost of Multi-AZ RDS and NAT Gateways	About two-thirds of the monthly bill in Table 7.04.	Single-AZ for pre-production; VPC endpoints for AWS services to reduce NAT traffic; reserved instances for steady-state production.

9.5 Future work

Beyond the improvements above, three extensions would make the system a more complete commerce platform. A fulfilment module would move orders from PAID to FULFILLED, with shipment tracking and the warehouse events that drive it. An asynchronous event backbone (Kafka or Amazon SQS) would decouple side effects such as confirmation emails, analytics and inventory synchronisation from the request path, and would be the natural place to publish order-placed and payment-succeeded events.

Finally, splitting the monolith along its existing module boundaries into separately deployable services would be justified once the catalogue read load and the order write load need to scale independently; the interfaces designed here are the seams along which that split would occur.

9.6 Closing remarks

The most valuable outcome of this project was not any single feature but the experience of making the pieces work together honestly: deciding where a transaction begins and ends, what happens when a dependency fails, how a third party's retries interact with internal state, and how to prove a performance claim. These are the questions that separate a demonstration from a service, and they are the questions this report has tried to answer.

Chapter 10: References

1. Spring Boot Reference Documentation, version 3.3. VMware Tanzu. https://docs.spring.io/spring-boot/docs/3.3.x/reference/html/ Accessed October 2026.
2. Spring Data JPA Reference Documentation. VMware Tanzu. https://docs.spring.io/spring-data/jpa/reference/ Accessed October 2026.
3. Spring Data Redis Reference Documentation. VMware Tanzu. https://docs.spring.io/spring-data/redis/reference/ Accessed October 2026.
4. Spring Framework Reference: Web MVC, HandlerInterceptor and @ControllerAdvice. https://docs.spring.io/spring-framework/reference/web/webmvc.html Accessed October 2026.
5. Hibernate ORM 6 User Guide. Red Hat. https://docs.jboss.org/hibernate/orm/6.5/userguide/html_single/Hibernate_User_Guide.html Accessed October 2026.
6. Redis Documentation: Data types (Hashes, Sets), EXPIRE, and Caching patterns. Redis Ltd. https://redis.io/docs/ Accessed October 2026.
7. MySQL 8.0 Reference Manual: InnoDB Locking and Transaction Model; Optimization and Indexes. Oracle. https://dev.mysql.com/doc/refman/8.0/en/ Accessed October 2026.
8. Razorpay API Reference: Payment Links; Webhooks and signature verification. Razorpay Software Pvt. Ltd. https://razorpay.com/docs/api/payments/payment-links/ Accessed October 2026.
9. Stripe API Reference: Checkout Sessions; Webhook signatures. Stripe, Inc. https://docs.stripe.com/api/checkout/sessions Accessed October 2026.
10. Jackson Databind documentation: Polymorphic type handling and PolymorphicTypeValidator. FasterXML. https://github.com/FasterXML/jackson-databind Accessed October 2026.
11. Niels Provos and David Mazieres, "A Future-Adaptable Password Scheme" (bcrypt), Proceedings of the USENIX Annual Technical Conference, 1999.
12. Amazon Web Services, Elastic Beanstalk Developer Guide: Java SE / Corretto platform; Rolling deployments. https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/ Accessed October 2026.
13. Amazon Web Services, Amazon VPC User Guide: Subnets, Route tables, NAT gateways, Security groups. https://docs.aws.amazon.com/vpc/latest/userguide/ Accessed October 2026.
14. Amazon Web Services, Amazon RDS User Guide: Multi-AZ deployments for MySQL. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/ Accessed October 2026.
15. Amazon Web Services, Amazon ElastiCache for Redis User Guide. https://docs.aws.amazon.com/AmazonElastiCache/latest/red-ug/ Accessed October 2026.
16. Martin Kleppmann, Designing Data-Intensive Applications, O'Reilly Media, 2017. Chapters 7 (Transactions) and 5 (Replication).
17. Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994. Strategy and Adapter patterns.
18. Robert C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall, 2017. Dependency inversion and boundaries.
19. Fake Store API documentation. https://fakestoreapi.com/docs Accessed October 2026.
20. Scaler Neovarsity backend specialization lecture notes and assignments on Spring Boot, JPA, Redis and payment gateways, 2025-2026. Instructor: Naman Bhalla.
21. OWASP Cheat Sheet Series: Password Storage; Deserialization. OWASP Foundation. https://cheatsheetseries.owasp.org/ Accessed October 2026.
22. Apache Maven Project, Maven Wrapper documentation. https://maven.apache.org/wrapper/ Accessed October 2026.

Note on tooling: an AI coding assistant (Kiro) was used during development for code generation, review and drafting of documentation, under the author's direction. All design decisions, verification and the final content are the author's own, in accordance with the declaration on page 3.